Abraham of London — Engineering Manual

Version: 3.2
Date: 1 July 2026
Classification: Internal — Engineering
Repository: `aol-check-visual`

Document Control

Field	Detail
Classification	Internal — Engineering
Version	3.2
Effective Date	July 2026
Review Cycle	Quarterly
Custodian	Lead Engineer
Next Review	October 2026

PREAMBLE

Abraham of London is a Decision Authority Infrastructure platform. It provides diagnostic, alignment, and governance tooling for individuals and enterprises making consequential decisions. The system scores decision conditions, detects contradictions, produces governed synthesis, and tracks decision memory over time.

The platform is a commercial delivery mechanism built on Next.js 16, TypeScript strict mode, PostgreSQL (Neon), and deployed to Netlify.

This document constitutes the definitive technical reference for all engineering work on the Abraham of London platform. It governs how the system is built, deployed, maintained, and extended. Every line of code committed to this repository is subject to the standards contained herein. This is not a set of suggestions or best practices to be weighed against developer preference. It is doctrine.

The platform's reliability, security, and capacity to scale depend on uniform adherence to these patterns. Deviations require written justification and approval from the Lead Engineer.

This manual is a living document, reviewed quarterly and updated as the platform evolves. However, its core architectural principles — separation of concerns, type safety, server-first rendering, content-as-data — are permanent. They will outlast any individual framework version.

HOW TO USE THIS MANUAL

Role-Based Reading Paths

Role	Required Reading
New Developer	Parts I-III (Architecture, Core Systems, Domain Systems)
Frontend Engineer	Parts II, V (Core Systems, Presentation Layer)
Backend Engineer	Parts II, III, VI (Core Systems, Domain, APIs)
DevOps / Platform	Parts VIII-IX (Operations, Security)
Full-Stack Engineer	All Parts sequentially

Conventions Used in This Document

Convention	Meaning
`monospace`	File paths, commands, code identifiers
Bold	Critical terms, model names, required values
`// ...`	Omitted code for brevity
Key Principle block	Foundational rule that must never be violated
Warning block	Common mistake or dangerous anti-pattern

> **KEY PRINCIPLE**
>
> When this document conflicts with external documentation (framework docs, blog posts, Stack Overflow), this document takes precedence for this codebase. External patterns are adopted only after validation against the architectural principles defined in Part I.

TABLE OF CONTENTS

Part 0 — Operating Doctrine

- Chapter 0: The Five-Cut Loop

Part I — System Architecture

- Chapter 1: Platform Overview
- Chapter 2: Technology Stack
- Chapter 3: Application Architecture

Part II — Core Systems

- Chapter 4: Database Layer
- Chapter 5: Authentication and Authorization
- Chapter 6: Content System

Part III — Domain Systems

- Chapter 7: Diagnostic Engine
- Chapter 8: Alignment & Campaign System
- Chapter 9: Strategy Room & Execution
- Chapter 10: Commercial Layer

Part IV — Product Elevation Layer

- Chapter 11: Cost of Inaction Engine
- Chapter 12: Execution Failure Predictor
- Chapter 13: Decision Authority Index
- Chapter 14: Decision Memory Service

Part V — Presentation Layer

- Chapter 15: Styling Architecture
- Chapter 16: Component Architecture
- Chapter 17: State Management

Part VI — API & Security Layer

- Chapter 18: API Architecture
- Chapter 19: Public DTO Contract
- Chapter 20: IP Protection Architecture
- Chapter 21: Rate Limiting
- Chapter 22: Auth Security

Part VII — Quality & Testing

- Chapter 23: Testing Strategy
- Chapter 24: Quality Gate
- Chapter 25: Definition of Clean
- Chapter 25A: Performance Standards
- Chapter 25B: Error Handling Philosophy
- Chapter 25C: Code Quality
- Chapter 25D: Monitoring Architecture

Part VIII — Operations & Deployment

- Chapter 26: Build Pipeline
- Chapter 27: Deployment
- Chapter 28: Environment Management
- Chapter 29: Monitoring

Part IX — Security

- Chapter 30: Security Architecture
- Chapter 31: Authentication Security
- Chapter 32: Data Protection
- Chapter 32A: ZTHVF Security Validation Framework

Part X — Canonical Contract Reference

- Chapter 33: Intelligence Spine Contract
- Chapter 34: Constitutional Derivation
- Chapter 35: Scoring API Contract

Part XI — PDF Pipeline

- Chapter 36: PDF Architecture
- Chapter 37: PDF Operations

Part XII — External Integrations

- Chapter 38: Integration Catalog

Part XIII — Serverless Functions

- Chapter 39: Netlify Functions

Part XIV — Developer Operations

- Chapter 40: Local Development Setup
- Chapter 41: Development Workflow
- Chapter 42: Scripts Reference
- Chapter 43: Troubleshooting Guide

Appendices

- Appendix A: Prisma Model Inventory
- Appendix B: API Route Inventory
- Appendix C: Environment Variable Reference

- Appendix D: Database Schema Extended Reference
- Appendix E: Quality Gate Checklist
- Appendix F: Architecture Decision Records
- Appendix G: Revision History
- Appendix H: Amendment Procedure

QUICK START

```
git clone <repo-url> && cd aol-check-visual
pnpm install
npx prisma generate
pnpm dev
```

Key Commands

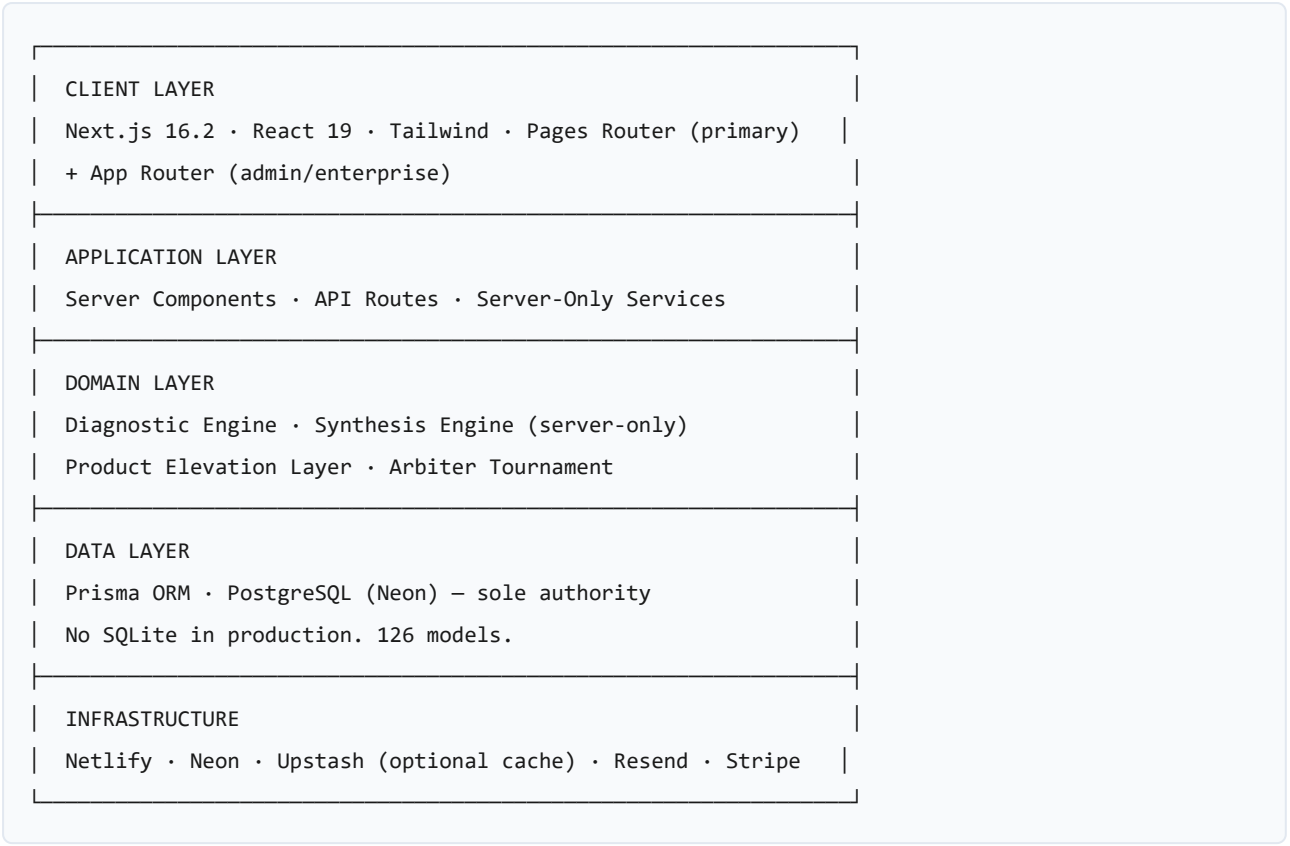
Command	Purpose
`pnpm dev`	Start development server (runs MDX gate + contentlayer build first)
`pnpm build`	Production build (generates EPUBs, then next build)
`pnpm quality:full`	Run `scripts/quality/full-validation.mjs` — the only acceptable clean bill
`pnpm test:unit`	Run Vitest unit + integration tests
`pnpm test:e2e`	Run Playwright end-to-end tests (separate runner)

Required .env values for local dev:

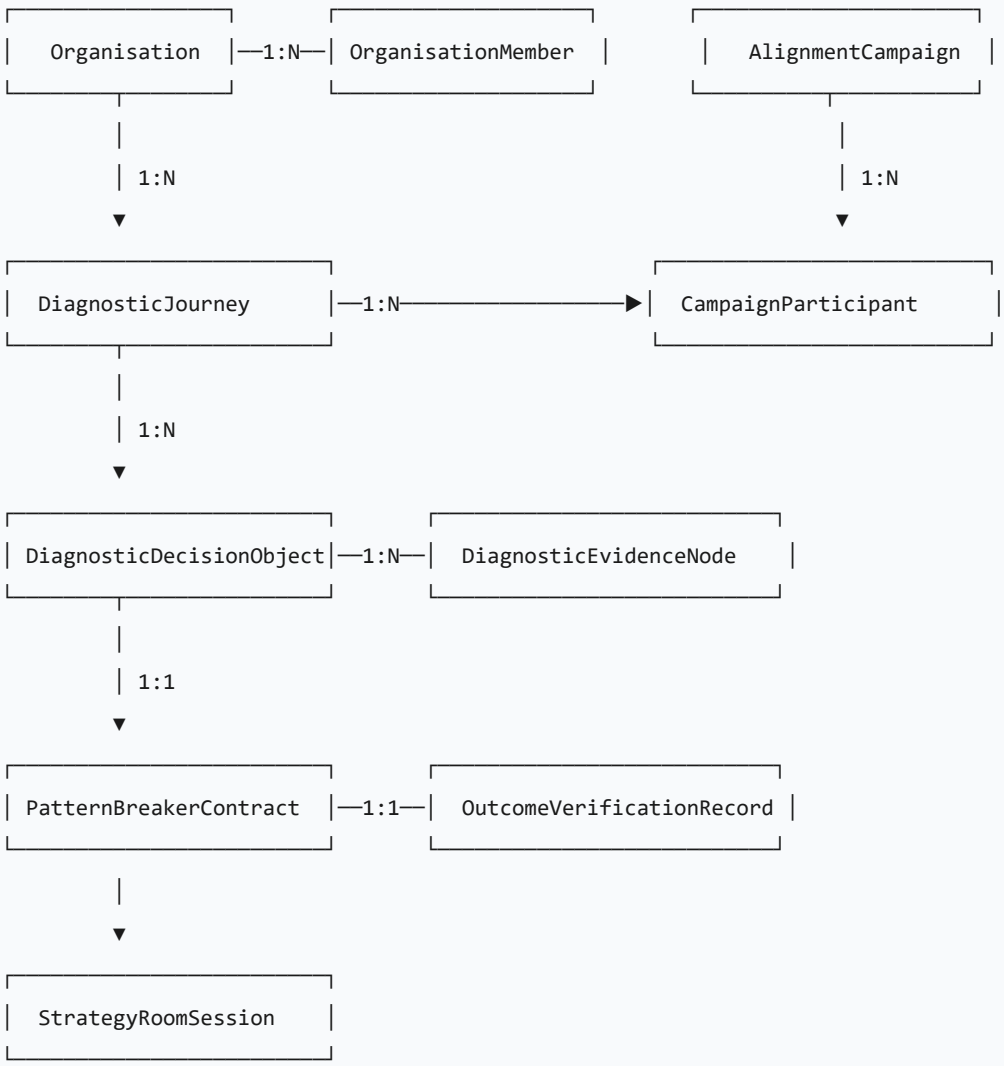
- `DATABASE_URL=postgresql://...` (Neon connection string or local PostgreSQL)
- `NEXTAUTH_SECRET=any-32-char-string`
- `NEXTAUTH_URL=http://localhost:3000`

SYSTEM DNA — DATA MODEL OVERVIEW

The platform is organized in five layers:


CLIENT LAYER
Next.js 16.2 · React 19 · Tailwind · Pages Router (primary)
+ App Router (admin/enterprise)
APPLICATION LAYER
Server Components · API Routes · Server-Only Services
DOMAIN LAYER
Diagnostic Engine · Synthesis Engine (server-only)
Product Elevation Layer · Arbiter Tournament
DATA LAYER
Prisma ORM · PostgreSQL (Neon) — sole authority
No SQLite in production. 126 models.
INFRASTRUCTURE
Netlify · Neon · Upstash (optional cache) · Resend · Stripe

Entity Relationship Diagram



System Status Map

System	Status	Notes
Diagnostic Engine	Production	Server-side scoring, arbiter validated
Multi-user Collision	Production	13 contradiction archetypes, 6-tier severity
Enforcement Engine	Production	Contracts + breach + escalation
Decision Ledger	Production	Implicit via journey model + credit score
Strategy Room	Production	Full state machine, 10+ API endpoints
Outcome Verification	Production	Confidence-capped, 5 classifications
Boardroom Mode	Production	Dossier builder + PDF; delivery state machine, entitlement, and web/PDF access under test coverage (`tests/product-estate/boardroom-*` , `tests/admin/boardroom-*` , `tests/reporting/boardroom-*`)
Case Study Pipeline	Production	Eligibility + draft builder; trust/gating covered by `tests/product/case-study-trust.test.ts`
AI Synthesis	Production	Claude/OpenAI integration with arbiter guard
Email System	Production	Resend + templates
PDF Pipeline	Production	5 paths, 198 assets, registry
North Star Metrics	Production	QER/DAR/TAR with hard thresholds (`lib/follow-up/north-star-metrics.ts`)
System Integrity Mode	Production	Global kill switch — degrades surfaces when metrics breach critical (`lib/follow-up/system-integrity-mode.ts`)
Founder Dashboard	Production	Operating data layer: sell/revenue/breakage/fix-now panels (`lib/analytics/founder-dashboard.ts`)
Redis Caching	Optional	Graceful degradation when unavailable

PART 0 — OPERATING DOCTRINE

Chapter 0: The Five-Cut Loop

Canonical charter: `docs/five-cut-loop-charter.md`

This chapter precedes Chapter 1 deliberately. It governs *how the requirements in every later chapter were derived*, and how every future change to this platform is reasoned about. Read it first; it outranks the implementation detail that follows.

> **KEY PRINCIPLE**
>
> The most common mistake of smart engineers is to optimise a thing that should not exist. So you do not optimise until after you have tried to delete. The order of the five cuts is doctrine.

The five cuts, in order

1. **Question the requirement.** Make the requirements less dumb. Every requirement is dumb to some degree, no matter how smart the person who set it. Each requirement must carry a **named human owner** — not "the system," not "the framework," not "we've always done it."
2. **Try to delete the part or step.** Deletion is the default; retention must be argued. **If you are not forced to add back at least 10% of what you delete, you did not delete enough.** A 0% add-back rate means you were too conservative.
3. **Simplify / optimise** — but only what survived Cut 2. Optimising a deletion-candidate is the cardinal sin this chapter exists to prevent.
4. **Accelerate** — but only what has been simplified. Speeding up accidental complexity entrenches it.
5. **Automate** — last. Automating a broken or unnecessary step makes it permanent.

Why the order matters here, concretely

This repository's history is the cautionary tale. Generations of one-off recovery scripts (`fix-hot-errors-v3` → `v4` → `v4_1` → `v4_2` , `ULTIMATE-FIX.ps1` , and dozens more — see `docs/repository-hygiene-ledger.md`) were excellent engineering aimed at the wrong question: they *automated and optimised* (Cuts 4–5) a recovery process for a state the build should never have entered. Cut 1 would have asked why the state existed; Cut 2 would have deleted the need. The scripts are the monument to skipping cuts 1 and 2.

Reconciliation with "extend, don't fragment"

Cut 2 is pointed at firefight residue and unowned requirements — **never** at the living core. The Living Intelligence layer (`lib/living-intelligence/*`), the EDOS decision spine, the active build configuration, and shared config/lockfiles have already survived deletion and are load-bearing; they are extended, never fragmented, never cut. Deleting *fragments* is precisely how this estate stays one living system. The two rules are the same instruction seen from two sides.

Enforcement

- Every implementation plan opens by answering the five cuts for its phase.
- Every PR completes the Five-Cut checklist (`.github/PULL_REQUEST_TEMPLATE.md`).
- The **Definition of Clean** (Chapter 25) includes "survived Cut 2."
- The monthly Five-Cut review reports the measured add-back rate and the net change in repository surface area.

PART I — SYSTEM ARCHITECTURE

Chapter 1: Platform Overview

Abraham of London is a full-stack Next.js 16 application written in TypeScript strict mode. It serves as a commercial delivery mechanism for decision authority diagnostics, alignment campaigns, strategy room sessions, and enterprise governance reporting.

The platform operates across two routing systems. The Pages Router (`pages/`) handles the primary user-facing experience: diagnostics, strategy room, content delivery, authentication, and the commercial surface. The App Router (`app/`) handles admin dashboards, enterprise campaign management, PDF rendering, purpose alignment, and the strategy room execution interface.

All scoring, classification, and synthesis logic runs server-side only. The client collects user input, submits it to API routes, and renders the public DTO that comes back. No thresholds, weights, classification rules, or scoring formulas exist in the client bundle.

The system enforces intellectual property protection through a layered architecture: public pages use controlled ambiguity ("proprietary scoring model," "multi-factor evaluation"), the API returns only sanitized DTOs, and the scoring engine imports `server-only` to prevent accidental client bundling.

Every diagnostic produces an IntelligenceSpine — a single contract object that flows through every stage from fast diagnostic through constitutional assessment, team validation, enterprise pricing, executive reporting, strategy room simulation, and outcome verification. No stage computes in isolation. No stage re-derives context from zero.

> **KEY PRINCIPLE**
>
> The platform exists to convert attention into revenue through structured confrontation. Every feature, every route, every component must trace its justification back to a stage in the product ladder defined in the Institutional Manual, Part II.

The Five Layers

Layer	Responsibility	Key Technologies	Location
Presentation	UI rendering, user interaction, visual state	React 19, Tailwind, Radix UI, CVA	`app/` , `pages/` , `components/`
Application	Request handling, orchestration, auth enforcement	Next.js middleware, API routes	`pages/api/` , `app/api/` , `middleware.ts`
Domain Logic	Business rules, scoring, synthesis, content processing	Pure TypeScript modules	`lib/` , `services/`
Data	Persistence, queries, schema, migrations	Prisma, PostgreSQL (Neon)	`prisma/` , `lib/prisma.server.ts`
Infrastructure	Hosting, external services, network	Netlify, Neon, Upstash, Resend, Stripe	`netlify.toml` , env vars

> **WARNING**
>
> Importing `@prisma/client` in any file that runs in the browser is a critical violation. Prisma is server-only. All database access must route through `lib/prisma.server.ts` and be consumed exclusively in Server Components, Server Actions, or API routes.

Directory Structure

```
C:\aol-check-visual\
├── app/                    → App Router: admin, enterprise, PDF, purpose alignment
│   ├── admin/              → Admin dashboard
│   ├── enterprise/         → Organisation-scoped routes
│   ├── __pdf/              → PDF rendering
│   ├── render/             → PDF render routes
│   ├── purpose-alignment/  → Purpose alignment assessment
│   ├── strategy-room/      → Strategy room execution
│   ├── downloads/          → Downloads management
│   ├── actions/            → Server Actions (form handlers, mutations)
│   ├── api/                → App Router API routes
│   └── layout.tsx          → Root layout with provider composition
├── pages/                  → Pages Router (primary user-facing)
│   ├── api/                → Primary API routes (diagnostics, auth, commerce)
│   │   ├── auth/           → NextAuth + identity endpoints
│   │   ├── diagnostics/    → Diagnostic scoring, submission, reports
│   │   ├── stripe/         → Payment processing
│   │   └── webhooks/       → Stripe webhook handlers
│   └── [page routes]       → Diagnostic flows, content, strategy room intake
├── components/             → Shared UI components
│   ├── ui/                 → Primitives (Button, Card, Input, Dialog)
│   ├── layout/             → Layout components (Header, Footer, Sidebar)
│   ├── features/           → Feature-specific composed components
│   ├── forms/              → Form compositions
│   ├── charts/             → Data visualization
│   ├── pdf/                → PDF-specific React components
│   └── shared/             → Cross-cutting (ErrorBoundary, Loading)
├── lib/                    → Domain logic, utilities, service clients
│   ├── prisma.server.ts    → Database singleton (SERVER ONLY)
│   ├── server/             → Server-only modules (scoring, auth, rate limiting)
│   ├── decision/           → Decision engine (CaseObject, synthesis, arbiter)
│   ├── diagnostics/        → Diagnostic pipeline modules
│   ├── auth/               → Auth utilities, guards, session helpers
│   ├── ai/                 → AI integration (Anthropic, OpenAI)
│   ├── email/              → Email templates and sending logic
│   ├── stripe/             → Payment processing
│   ├── pdf/                → PDF registry, governance, watermarking
│   └── validation/         → Zod schemas
├── prisma/                 → Database schema, migrations, seed
│   ├── schema.prisma       → The single source of truth for data models (126 models)
│   ├── migrations/         → Production migration history
│   └── seed.ts             → Development data seeding
├── content/                → MDX content files (processed by Contentlayer2)
```

```
│   ├── briefs/             → Decision briefs
│   ├── blog/               → Blog posts
│   ├── vault/              → Premium gated content
│   └── registry/           → Institutional registry entries
├── public/                 → Static assets (images, fonts, favicons)
├── styles/                 → Global CSS, Tailwind base layer
├── types/                  → Shared TypeScript type definitions
├── scripts/                → Build scripts, content processing, utilities
├── netlify/                → Netlify functions
│   └── functions_src/      → Serverless function source
├── _templates/             → Code generation templates
├── _tests_/                → Test suites (Vitest unit, Playwright E2E)
├── middleware.ts           → Edge middleware (auth, redirects, rate limiting)
├── contentlayer.config.ts  → Contentlayer2 document type definitions
├── tailwind.config.cjs     → Tailwind configuration with design tokens
├── next.config.ts          → Next.js configuration
├── tsconfig.json           → TypeScript strict configuration
├── netlify.toml            → Netlify deployment configuration
└── package.json            → Dependencies and scripts
```

> **KEY PRINCIPLE**
>
> New files must be placed according to this structure without exception. If a file does not have an obvious home, it belongs in `lib/` with a descriptive filename. Creating new top-level directories requires Lead Engineer approval.

Module Boundaries

The following import rules are enforced:

Source	May Import From	Must Never Import From
`app/`	`components/`, `lib/`, `types/`	`prisma/` directly, `node_modules` internals
`components/`	Other `components/`, `lib/`, `types/`	`app/`, `prisma/`, `pages/`
`lib/`	Other `lib/`, `types/`, external packages	`app/`, `components/`, `pages/`
`pages/api/`	`lib/`, `types/`, `prisma/` (via server module)	`app/`, `components/`
`prisma/`	Nothing (leaf module)	Everything else

Chapter 2: Technology Stack

Runtime

- **Next.js:** 16.2.2

- **React:** 19
- **TypeScript:** 5.9.3, strict mode with `noUncheckedIndexedAccess`, `noImplicitOverride`, `noFallthroughCasesInSwitch`
- **Node.js:** >=20.0.0
- **Package Manager:** pnpm >=9.0.0

Complete Dependency Catalog

Runtime Framework

Package	Version	Purpose
`next`	16.2.2	Full-stack React framework, App Router, API routes
`react`	19	UI library, Server Components, Suspense
`react-dom`	19	DOM rendering
`typescript`	5.9.3	Type system (strict mode enforced)

Database and ORM

Package	Version	Purpose
`prisma`	6.6.0	Schema definition, migrations, CLI
`@prisma/client`	6.6.0	Type-safe database client
`@neondatabase/serverless`	—	Serverless PostgreSQL driver (production)

Authentication

Package	Version	Purpose
`next-auth`	4.24.13	Authentication framework
`@auth/prisma-adapter`	—	Prisma session/account storage
`argon2`	—	Password hashing (Credentials provider)
`otplib`	—	TOTP generation and verification (MFA)

Styling and UI

Package	Version	Purpose
`tailwindcss`	3.4.17	Utility-first CSS framework
`@radix-ui/*`	—	Accessible headless UI primitives
`class-variance-authority`	—	Component variant management (CVA)
`clsx + tailwind-merge`	—	Conditional class composition

Content Processing

Package	Version	Purpose
`contentlayer2`	—	MDX processing, type generation
`mdx`	—	Markdown + JSX authoring
`gray-matter`	—	Frontmatter parsing
`remark-gfm`	—	GitHub Flavored Markdown
`rehype-pretty-code`	—	Syntax highlighting
`rehype-slug`	—	Heading anchors
`rehype-autolink-headings`	—	Clickable heading links

Email

Package	Version	Purpose
`resend`	6.9.3	Primary email delivery
`nodemailer`	—	Fallback email transport

PDF Generation

Package	Version	Purpose
`@react-pdf/renderer`	—	React-based PDF composition
`jspdf`	—	Programmatic PDF generation
`puppeteer`	—	HTML-to-PDF via headless Chrome
`md-to-pdf`	—	Markdown-to-PDF conversion

AI Integration

Package	Version	Purpose
`@anthropic-ai/sdk`	—	Claude API (primary AI provider)
`openai`	—	OpenAI API (secondary)

Payments

Package	Version	Purpose
`stripe`	—	Payment processing, subscriptions, webhooks

Search

Package	Version	Purpose
`algoliasearch`	—	Full-text search (production)
`fuse.js`	—	Client-side fuzzy search (fallback)

Security

Package	Version	Purpose
`@upstash/ratelimit`	—	Distributed rate limiting
`zod`	—	Runtime schema validation
`dompurify`	—	HTML sanitization

Testing

Package	Version	Purpose
`vitest`	4.1.2	Unit and integration testing
`@playwright/test`	—	End-to-end browser testing

Database

PostgreSQL via Neon is the authoritative data store for ALL persistent data. The Prisma schema (`prisma/schema.prisma`) is the single source of truth with 126 models. There is no SQLite in production. Legacy `db:sqlite:*` scripts exist in `package.json` but are not part of the production data path.

Cache

Redis via Upstash is OPTIONAL. The system functions without it. When Redis is unavailable, rate limiting falls back to PostgreSQL (the `RateLimitBucket` model). If both are unavailable on critical routes, the system fails closed (request denied).

ORM

Prisma with `schema.prisma` as the single source of truth. The generated client outputs to `node_modules/.prisma/client` . Migrations use `prisma migrate dev` for development and `prisma migrate deploy` for production.

Deployment

Netlify. Not Vercel. The `netlify.toml` configures the build command (`pnpm build:netlify`), Node 20.20.0, and the `@netlify/plugin-nextjs` adapter. The publish directory is `.next` .

Email

Resend for transactional email delivery.

Payments

Stripe for checkout, entitlement verification, and report purchases.

Security

- **Input validation:** zod schemas on all API routes

- **Sanitization:** DOMPurify for user-generated content
- **Encryption:** AES-256-GCM for spine data in sessionStorage (per-session key via Web Crypto API)
- **Server-only scoring:** `import "server-only"` on all scoring and synthesis modules
- **Rate limiting:** Redis primary, Postgres fallback, fail-closed on critical routes

Version Pinning Policy

All dependencies use **exact versions** in `package.json` (no `^` or `~` prefixes). This eliminates supply-chain drift and ensures reproducible builds across all environments.

```
{
  "dependencies": {
    "next": "16.2.2",      // Exact
    "react": "19",         // Exact
    "next": "^16.2.2"      // PROHIBITED — semver range
  }
}
```

> **WARNING**
>
> Running `pnpm update` without explicit package names is prohibited. Bulk updates bypass review and can introduce breaking changes. Every dependency upgrade must be individually justified, tested, and committed separately.

Upgrade Strategy

1. **Patch versions** (e.g., 16.2.1 to 16.2.2) — Apply after verifying changelog. No approval required.
2. **Minor versions** (e.g., 16.2.x to 16.3.0) — Apply in dedicated branch. Run full test suite. Requires code review.
3. **Major versions** (e.g., 16.x to 17.0) — Requires Lead Engineer approval, dedicated migration branch, Architecture Decision Record (ADR), and phased rollout.

Compatibility Matrix

Component	Required Version	Notes
Node.js	>= 20.0.0	LTS releases only. Node 22 preferred.
pnpm	>= 10.33	Workspace protocol, strict peer deps
Next.js	16.2.2	Pages Router primary, App Router for admin
React	19	Server Components, `use` hook, Actions
TypeScript	5.9.3	`strict: true`, `noUncheckedIndexedAccess: true`
Prisma	6.6.0	Client extensions, typed JSON
PostgreSQL	>= 15	Neon serverless (production)

Prohibited Dependencies

The following categories of packages must never be added:

Category	Reason	Use Instead
jQuery	Incompatible with React model	Native DOM via refs
Moment.js	Enormous bundle, deprecated	`date-fns` or native `Intl`
Lodash (full)	Tree-shaking issues	Individual `lodash-es/*` imports or native
Express	Next.js handles routing	API routes
Mongoose	We use PostgreSQL + Prisma	Prisma client
Firebase	Vendor lock-in, architectural conflict	Neon + Upstash
Zustand	Architecture uses React state only	`useState`, `useEffect`, `useReducer`
Redux	Architecture uses React state only	React built-in state
React Query	Architecture uses React state only	Server Components + API routes

---

Chapter 3: Application Architecture

Routing

Pages Router (`pages/`) is the primary router. It handles:

- Fast diagnostic flow and submission
- Constitutional intake and reports
- Strategy room intake
- Content pages (briefs, canon, books, blog, artifacts)
- Authentication (`pages/api/auth/[...nextauth].ts`)
- All diagnostic API routes (`pages/api/diagnostics/`)
- Commercial endpoints (Stripe checkout, downloads)

App Router (`app/`) handles:

- Admin dashboard (`app/admin/`)
- Enterprise campaign management (`app/enterprise/`)
- PDF rendering (`app/__pdf/` , `app/render/`)
- Purpose alignment assessment (`app/purpose-alignment/`)
- Strategy room execution sessions (`app/strategy-room/`)
- Downloads management (`app/downloads/`)
- Registry and settings

Route Groups

The App Router uses route groups for layout isolation:

- `(dashboard)` — authenticated dashboard shell
- `admin` — admin-only routes
- `enterprise` — organisation-scoped routes

Rendering Strategies

Strategy	Use Case	Configuration
SSR (default)	Dynamic pages, personalised content, auth-gated	Default for all pages
SSG	Static content pages, blog posts, briefs	`generateStaticParams()` + no dynamic data
ISR	Semi-static content that updates periodically	`revalidate: 3600` (1 hour default)
Client	Interactive widgets, forms, real-time UI	`'use client'` directive

Default rule: Everything is SSR unless there is a specific reason to choose otherwise. SSR provides the best balance of performance, SEO, and data freshness.

ISR configuration:

```
// For content pages (blog, briefs, registry)
export const revalidate = 3600; // Re-generate every hour

// For highly dynamic pages
export const revalidate = 0; // Always fresh (equivalent to SSR)

// For truly static pages (legal, about)
export const dynamic = 'force-static';
```

- **Static Generation:** Content pages (briefs, canon, books) via Contentlayer2
- **Server-Side Rendering:** Diagnostic results, authenticated pages
- **Client Components:** Interactive diagnostic forms, conversion surfaces
- **Server Components:** Admin views, report rendering, data-heavy pages

Provider Composition

The root layout composes providers. The application uses minimal providers — no global state library providers needed.

```
// app/layout.tsx — Provider hierarchy (outermost → innermost)
<AuthProvider>              // 1. Session context (NextAuth)
  <ThemeProvider>           // 2. Light/dark mode
    {children}
  </ThemeProvider>
</AuthProvider>
```

> **WARNING**
>
> Do not add new providers without considering the render cost. Each provider wrapping `{children}` creates a potential re-render boundary. New providers require Lead Engineer approval.

Server Components vs Client Components

Server Components (default — no directive needed):

- Access database directly via Prisma
- Read environment variables (server secrets)
- Use `async/await` at the component level
- Zero JavaScript sent to client
- Cannot use `useState`, `useEffect`, event handlers, or browser APIs

Client Components (`'use client'` directive at top of file):

- Interactive UI (forms, modals, toggles)
- Browser API access (localStorage, geolocation)
- Event handlers (onClick, onChange, onSubmit)
- React hooks (useState, useEffect, useRef)
- Third-party libraries that use browser APIs

Decision criteria — use a Client Component only when:

1. The component requires interactivity (user events)
2. The component uses React hooks that depend on client state
3. The component imports a library that accesses `window` or `document`

If none of these apply, it must remain a Server Component.

```
// CORRECT — Server Component (default)
// app/(public)/about/page.tsx
import { getFounderBio } from '@/lib/content';

export default async function AboutPage() {
  const bio = await getFounderBio();
  return <FounderProfile data={bio} />;
}

// CORRECT — Client Component (interactive)
// components/ui/ThemeToggle.tsx
'use client';
import { useState } from 'react';

export function ThemeToggle() {
  const [dark, setDark] = useState(false);
  return <button onClick={() => setDark(!dark)}>Toggle</button>;
}
```

> **KEY PRINCIPLE**
>
> The boundary between Server and Client Components is an architectural decision, not a convenience toggle. Push the `'use client'` boundary as deep into the component tree as possible. A page should be a Server Component that renders Client Component leaves — never the reverse.

Server Actions

Server Actions (`app/actions/`) are async functions that run on the server and can be called directly from Client Components without creating API endpoints.

```
// app/actions/assessment.ts
'use server';

import { z } from 'zod';
import { getAuthSession } from '@/lib/auth';
import { prisma } from '@/lib/prisma.server';

const SubmitSchema = z.object({
  assessmentId: z.string().uuid(),
  answers: z.array(z.object({
    questionId: z.string(),
    value: z.number().min(1).max(5),
  })),
});

export async function submitAssessment(formData: FormData) {
  const session = await getAuthSession();
  if (!session) throw new Error('Unauthorized');

  const parsed = SubmitSchema.parse(Object.fromEntries(formData));
  // ... process submission
}
```

Rules for Server Actions:

1. Always validate input with Zod before processing
2. Always check authentication/authorization
3. Never return sensitive data (internal IDs, secrets)
4. Use `revalidatePath()` or `revalidateTag()` after mutations
5. Throw errors for invalid states — the framework handles error UI

Proxy (Institutional Perimeter)

The system uses `proxy.ts` at the project root as its **Institutional Perimeter (V5.1)**. This replaces the standard Next.js `middleware.ts` pattern. It exports a `proxy()` function and a `config.matcher` that intercepts all requests except static assets.

Execution order:

Step	Check	Action on Failure
0	PDF download guard	Redirects `/assets/downloads/*.pdf` → `/api/downloads/{slug}` (307)
1	Dev bypass	`BYPASS_SOVEREIGN=true` in development only
2	Internal bypass	`X-Directorate-Bypass` header matches `INTERNAL_BYPASS_KEY`
3	Canonical host redirect	Non-canonical hostname → 308 to `www.abrahamoflondon.org`
4	Global lockdown	Checks `/api/system/lock-status` — non-admin users get 503
5	Public path bypass	Matches `PUBLIC_PREFIXES` — passes through with security headers
6	Rate limiting	Per-IP + pathname, tiered by route type. 429 on limit exceeded
7	Constitutional authority	For constitutional paths — requires sovereign session + authority level
8	Admin IP restriction	`ADMIN_ALLOWED_IPS` check for admin paths
9	Session & role validation	NextAuth JWT + access cookie. Admin requires role hierarchy
10	Auth tier enforcement	Edge-safe tier: Tier 0 (public) → Tier 3 (architect)
11	Security headers	CSP, HSTS, COOP, CORP, X-Frame-Options, Permissions-Policy

Auth tier classification:

Tier	Level	Route Prefixes
Tier 3	architect (admin)	`/inner-circle/admin`, `/api/admin`, `/directorate`
Tier 2	inner_circle	`/inner-circle`, `/private`, `/vault`, `/board`
Tier 1	member	`/consulting`, `/strategy`
Tier 0	public	All `PUBLIC_PREFIXES`

```
// proxy.ts — matcher covers all non-static routes
export const config = {
  matcher: [
    "/((?!_next/static|_next/image|favicon.ico|robots.txt|sitemap.xml).*)",
  ],
};
```

> **WARNING**
>
> The proxy runs on every matched request at the Edge. It must remain fast. Database queries are limited to lockdown checks (cached 15s). Auth uses JWT verification only (no DB round-trip). Constitutional authority validation uses in-memory caches with 5-minute TTL.

> **KEY PRINCIPLE**

> Routes not covered by proxy tier enforcement (e.g., `/api/diagnostics/*`, `/api/strategy-room/*`, `/api/billing/*`) must implement their own auth guards. The proxy provides security headers and rate limiting but NOT auth gating for these routes.

PART II — CORE SYSTEMS

Chapter 4: Database Layer

PostgreSQL (Neon) is the sole persistent data authority. The Prisma schema contains 126 models organized into the following domains:

Prisma Architecture

Prisma 6.6.0 serves as the ORM and schema management layer. The Prisma client is instantiated as a singleton in `lib/prisma.server.ts`:

```
// lib/prisma.server.ts
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma ||
  new PrismaClient({
    log: process.env.NODE_ENV === 'development' ? ['query', 'warn', 'error'] : ['error'],
  });

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

> **KEY PRINCIPLE**
>
> The singleton pattern prevents connection exhaustion during development (hot reloading creates new module instances). In production, Next.js standalone output ensures a single instance. Never instantiate `PrismaClient` anywhere other than this file.

Schema Overview — Domain Groupings

Identity & Access

`User`, `Account`, `VerificationToken`, `Entitlement`, `AccessKey`, `AccessKeyUse`, `AccessAuditLog`, `AccessInvite`, `Session`, `AdminSession`, `ApiKey`, `ApiLog`, `InnerCircleKey`, `MfaSetup`, `SecurityLog`

Organisation & Campaigns

`Organisation`, `AlignmentCampaign`, `AlignmentSnapshot`, `OrganisationInvite`, `OrganisationMembership`, `CampaignParticipant`, `EnterpriseAssessment`, `TeamAssessmentSnapshot`, `TeamAssessmentCampaign`, `TeamAssessmentInvite`, `TeamAssessmentResponse`, `TeamAssessmentAggregate`, `OrganisationAssessmentSnapshot`, `LeadershipGapSnapshot`, `EnterpriseReport`, `ExecutiveReportingRun`, `ExecutiveReportingArtifact`

Diagnostics

`DiagnosticRecord`, `DiagnosticReportOrder`, `DiagnosticArtifact`, `DiagnosticRegenerationJob`, `DiagnosticAuditEvent`, `DiagnosticArtifactAccessGrant`, `DiagnosticLineageEvent`, `DiagnosticJourney`, `DiagnosticEvidenceNode`, `DiagnosticDecisionObject`, `DiagnosticStageRecord`, `DiagnosticThreadSnapshot`, `ConstitutionalIntakeReport`

Decision Infrastructure

`DecisionMemory`, `DecisionDependency`, `DecisionStakeholder`, `StakeholderPosition`, `DecisionRecommendationSession`, `DecisionRecommendationImpression`, `DecisionRecommendationClick`, `DecisionRecommendationConversion`, `DecisionSessionFollowup`, `DecisionAssetEfficacy`, `DecisionAssetPerformance`, `DecisionAssetContextPerformance`, `DecisionAssetGovernanceRule`, `DecisionSignalRegistry`, `DecisionGovernanceAlert`, `DecisionJourneyEvent`, `ConsequenceTimeline`, `EscalationEvent`, `CalibrationState`, `CalibrationEvent`, `PatternBreakerContract`

Strategy & Governance

`StrategyRoomSession`, `StrategyRoomRecommendationImpression`, `StrategyRoomFollowup`, `StrategyRoomConversion`, `StrategyRoomExecutionSession`, `StrategyDecisionLog`, `StrategyInquiry`, `StrategyIntake`, `StrategicIntervention`, `CorrectionNode`, `EnforcementPlaybook`, `PlaybookApplication`, `EnforcementCycle`, `RetainerContract`, `RetainedDecision`

Operations & Monitoring

`RateLimitBucket`, `RateLimitLog`, `GovernanceLog`, `GovernanceMetricDefinition`, `OperationalIncident`, `ArtifactManifest`, `JobDeadLetter`, `ServiceLevelSnapshot`, `RunbookEntry`, `MonitoringSnapshot`, `BenchmarkFact`, `BenchmarkCohortSnapshot`, `SystemAuditLog`, `AuditEvent`, `AuditResponse`, `FoundationTelemetryEvent`

Content & Commerce

`ContentMetadata`, `Framework`, `StrategicLink`, `ContentRelation`, `PrivateAnnotation`, `CanonEntry`, `StrategicFramework`, `DownloadAuditEvent`, `PrintAsset`, `PremiumDownloadToken`, `PremiumDownloadAttempt`, `FrameworkAccessLog`, `PageView`, `UserPreference`, `UserIntegration`, `InnerCircleMember`, `BillingCustomer`, `ClientEntitlement`, `ProofEvidence`, `FailedEntitlementGrant`

Purpose Alignment

`PurposeAlignmentAssessment`, `PurposeAlignmentReport`, `PurposeAlignmentReminderPreference`, `PurposeAlignmentReminderLog`

Deal Flow

`DealFlowSubmission`

Key Enumerations

```
enum Role {
  USER          // Default — public access only
  ADMIN         // Platform administration
  OWNER         // Founder — unrestricted access
}

enum AccessTier {
  PUBLIC        // Available to all visitors
  MEMBER        // Requires free account
  INNER_CIRCLE  // Requires paid membership
  RESTRICTED    // Requires specific entitlement
  CLIENT        // Active paying client only
  ARCHITECT     // Senior client tier
  OWNER         // Founder-only content
  TOP_SECRET    // System internals, never exposed
}

enum AlignmentBand {
  ALIGNED       // 75-100 — coherent, functional, generative
  DRIFTING      // 50-74  — losing coherence, early intervention warranted
  MISALIGNED    // 25-49  — structural contradiction, active harm possible
  DISORDERED    // 0-24   — systemic breakdown, urgent intervention required
}

enum AlignmentDomain {
  IDENTITY
  DECISION
  ENVIRONMENT
  BEHAVIOUR
  EMOTIONAL_ORDER
  LEGACY
}
```

Migration Strategy

Environment	Command	Behaviour
Development	`pnpm prisma db push`	Applies schema changes directly, no migration file
Preview	`pnpm prisma migrate deploy`	Applies pending migrations from `prisma/migrations/`
Production	`pnpm prisma migrate deploy`	Applies pending migrations (CI/CD pipeline only)

Creating a new migration:

```
pnpm prisma migrate dev --name descriptive_name
```

Rules:

1. Never edit a migration file after it has been committed
2. Never delete a migration file — use a new migration to reverse changes
3. Migration names must be descriptive: `add_mfa_fields_to_user`, not `update_schema`
4. Destructive migrations (dropping columns/tables) require explicit `-- WARNING: DESTRUCTIVE` comment

Migration Commands

```
pnpm db:generate    # prisma generate
pnpm db:push        # prisma db push (schema sync without migration)
pnpm db:migrate     # prisma migrate dev (create migration)
pnpm db:deploy      # prisma migrate deploy (apply in production)
pnpm db:seed        # tsx prisma/seed.ts
pnpm db:studio      # prisma studio (browser GUI)
pnpm db:reset       # prisma migrate reset --force && pnpm db:seed
```

Seeding

The seed script (`prisma/seed.ts`) populates the development database with representative data:

```
// prisma/seed.ts
import { PrismaClient } from '@prisma/client';
import { hash } from 'argon2';

const prisma = new PrismaClient();

async function main() {
  await prisma.user.upsert({
    where: { email: 'admin@abrahamoflondon.org' },
    update: {},
    create: {
      email: 'admin@abrahamoflondon.org',
      name: 'Admin',
      hashedPassword: await hash('dev-password-only'),
      role: 'OWNER',
    },
  });
  // Create sample assessments, content, etc.
}

main()
  .catch(console.error)
  .finally(() => prisma.$disconnect());
```

Server-Only Import Rule

> **KEY PRINCIPLE**
>
> The Prisma client must never be imported in code that could execute in the browser. This includes Client Components, utility functions imported by Client Components, or any file without explicit server-only guarantees.

Enforcement mechanisms:

1. The singleton lives in `lib/prisma.server.ts` — the `.server.ts` suffix triggers Next.js to error if imported from a client bundle
2. ESLint rules flag `@prisma/client` imports outside approved paths
3. The build will fail if Prisma appears in client chunks

Chapter 5: Authentication & Authorization

Identity Authority

NextAuth with JWT is the PRIMARY identity authority. The configuration lives in `pages/api/auth/[...nextauth].ts`. JWT tokens carry user ID, email, and role. The `NEXTAUTH_SECRET` environment variable is required.

```
export const authOptions: NextAuthOptions = {
  providers: [GoogleProvider, GitHubProvider, CredentialsProvider],
  adapter: PrismaAdapter(prisma),
  session: { strategy: 'jwt', maxAge: 30 * 24 * 60 * 60 }, // 30 days
  jwt: { maxAge: 30 * 24 * 60 * 60 },
  pages: {
    signIn: '/auth/signin',
    error: '/auth/error',
  },
  callbacks: { jwt, session, signIn },
};
```

OAuth Flow

Google OAuth:

- Scopes: `openid`, `email`, `profile`
- Used by: Public users, enterprise clients
- Auto-creates `User` + `Account` records on first sign-in

GitHub OAuth:

- Scopes: `read:user`, `user:email`
- Used by: Developers, internal team
- Links to existing account if email matches

Flow:

1. User clicks "Sign in with Google/GitHub"
2. Redirect to provider consent screen
3. Provider redirects back with authorization code
4. NextAuth exchanges code for tokens
5. User profile extracted, `User` upserted, `Account` linked
6. JWT issued with user ID, role, email
7. Redirect to callback URL or dashboard

Credentials Flow

The Credentials provider enables email/password authentication for admin users:

```
CredentialsProvider({
  name: 'credentials',
  credentials: {
    email: { type: 'email' },
    password: { type: 'password' },
    totp: { type: 'text' }, // MFA code (optional)
  },
  async authorize(credentials) {
    const user = await prisma.user.findUnique({
      where: { email: credentials.email },
    });
    if (!user?.hashedPassword) return null;

    const valid = await verify(user.hashedPassword, credentials.password);
    if (!valid) return null;

    // MFA check if enabled
    if (user.mfaSecret) {
      if (!credentials.totp) throw new Error('MFA_REQUIRED');
      const totpValid = authenticator.verify({
        token: credentials.totp,
        secret: user.mfaSecret,
      });
      if (!totpValid) throw new Error('MFA_INVALID');
    }

    return { id: user.id, email: user.email, role: user.role };
  },
})
```

> **WARNING**
>
> The Credentials provider is restricted to admin/owner accounts. Public users must authenticate via OAuth.

JWT Structure and Claims

```
interface JWTPayload {
  sub: string;          // User ID
  email: string;        // User email
  name: string;         // Display name
  role: Role;           // USER | ADMIN | OWNER
  tier: AccessTier;     // Highest access tier
  iat: number;          // Issued at
  exp: number;          // Expiry (30 days from issue)
}
```

The JWT is signed with `NEXTAUTH_SECRET` (HS256). It is stored in an HTTP-only cookie and refreshed on each request (sliding window).

Session Access Patterns

```
// Server Component / Server Action / API Route
import { getAuthSession } from '@/lib/auth';

export default async function ProtectedPage() {
  const session = await getAuthSession();
  if (!session) redirect('/auth/signin');
  return <Dashboard user={session.user} />;
}

// Client Component
'use client';
import { useSession } from 'next-auth/react';

export function UserMenu() {
  const { data: session, status } = useSession();
  if (status === 'loading') return <Skeleton />;
  if (!session) return <SignInButton />;
  return <Avatar user={session.user} />;
}
```

Cookie Architecture

- **`aol_access`**: Session presence indicator ONLY. It does NOT upgrade the user's tier. Its existence means "a session was established" but the tier must be resolved server-side via Prisma.
- **`next-auth.session-token`** / **`__Secure-next-auth.session-token`**: The actual NextAuth JWT. This is the sole identity credential.

Cookie Configuration

```
cookies: {
  sessionToken: {
    name: '__Secure-next-auth.session-token',
    options: {
      httpOnly: true,       // No JavaScript access
      sameSite: 'lax',      // CSRF protection
      path: '/',
      secure: true,         // HTTPS only (production)
      domain: '.abrahamoflondon.com',
    },
  },
}
```

> **KEY PRINCIPLE**
>
> Authentication cookies are always `httpOnly` and `secure` in production. The `__Secure-` prefix is a browser-enforced signal that the cookie was set over HTTPS.

Identity Resolution

`lib/server/auth/tokenStore.postgres.ts` exports `getSessionContext()` and `verifySession()`. These functions:

1. Decode the NextAuth JWT via `getToken()` from `next-auth/jwt`
2. Look up the user in Prisma via `getUserAccess()`
3. Return the resolved tier, role, and session metadata

The cookie alone does not determine tier. The server always resolves tier from the database.

Role System

Role	Capabilities	Assignment
USER	Public content, own assessments, own dashboard	Default on sign-up
ADMIN	All USER + user management, content admin, system config	Manual promotion
OWNER	All ADMIN + financial data, delete operations, system override	Bootstrap only

Roles are **hierarchical** — OWNER includes all ADMIN permissions, which includes all USER permissions.

Tier Hierarchy

```
public < member < inner_circle < restricted < client < architect < owner < top_secret
```

Each tier includes access to all content at its level and below. Tier assignment is managed through subscription status, manual assignment, and engagement status.

Guard Functions

```
// lib/auth/guards.ts

export async function requireAuth() {
  const session = await getAuthSession();
  if (!session) throw new AuthError('UNAUTHENTICATED');
  return session;
}

export async function requireRole(minimumRole: Role) {
  const session = await requireAuth();
  const hierarchy = { USER: 0, ADMIN: 1, OWNER: 2 };
  if (hierarchy[session.user.role] < hierarchy[minimumRole]) {
    throw new AuthError('INSUFFICIENT_ROLE');
  }
  return session;
}

export async function requireTier(minimumTier: AccessTier) {
  const session = await requireAuth();
  const tierOrder = [
    'PUBLIC', 'MEMBER', 'INNER_CIRCLE', 'RESTRICTED',
    'CLIENT', 'ARCHITECT', 'OWNER', 'TOP_SECRET'
  ];
  if (tierOrder.indexOf(session.user.tier) < tierOrder.indexOf(minimumTier)) {
    throw new AuthError('INSUFFICIENT_TIER');
  }
  return session;
}
```

Multi-Factor Authentication

MFA uses TOTP (Time-based One-Time Password) via the `otplib` library:

1. **Enrollment:** User generates secret, stored encrypted in `User.mfaSecret`
2. **QR Code:** Secret encoded as `otpauth://` URI, rendered as QR for authenticator app
3. **Verification:** On login, user provides 6-digit code, verified against secret with 30s window
4. **Recovery:** Backup codes generated at enrollment, hashed and stored

MFA is **required** for ADMIN and OWNER roles. USER role may optionally enable it.

Parameter	Value
Algorithm	SHA1 (TOTP standard)
Digits	6
Period	30 seconds
Window	1 (allows +/- 1 period drift)

Admin Bootstrap

First-time deployment uses the `BOOTSTRAP_ADMIN_EMAILS` environment variable:

```
BOOTSTRAP_ADMIN_EMAILS="info@abrahamoflondon.org,admin@abrahamoflondon.org,seunadaramola@gmail.com,ab
```

When a user with a matching email signs in for the first time, they are automatically assigned the OWNER role.

> **WARNING**
>
> Remove or clear `BOOTSTRAP_ADMIN_EMAILS` after initial setup is complete.

Rate Limiting

Priority chain:

1. **Redis (Upstash)** — primary store via `getRedis()`
2. **PostgreSQL** — fallback via `RateLimitBucket` model and `rate-limit-store.postgres.ts`
3. **Fail-closed** — if both are unavailable on critical routes, the request is denied

There is no in-memory rate limiting in production.

Rate limit configurations are defined in `lib/server/rate-limit-unified.ts`:

Key	Limit	Window
PUBLIC	120	60s
AUTH	60	60s
ADMIN	100	60s
API_STRICT	30	60s
API_GENERAL	100	1hr
INNER_CIRCLE_UNLOCK	30	10min
CONTACT	5	1hr
DOWNLOAD	20	1hr

Dev-Login

The `pages/api/auth/sovereign-login.ts` route exists for development convenience. It must return 404 in production. No development authentication bypass is permitted in deployed environments.

Chapter 6: Content System

Contentlayer2

Content is managed via Contentlayer2, configured in `contentlayer.config.ts`. The system processes MDX files from the `content/` directory and generates typed document definitions in `.contentlayer/generated/`.

```
content/*.mdx → Contentlayer2 → .contentlayer/generated/ → import in components
```

Document Type Definitions

The platform defines 20+ document types. Key types include:

```
// contentlayer.config.ts (simplified)
export const Brief = defineDocumentType(() => ({
  name: 'Brief',
  filePathPattern: 'briefs/**/*.mdx',
  contentType: 'mdx',
  fields: {
    title: { type: 'string', required: true },
    description: { type: 'string', required: true },
    date: { type: 'date', required: true },
    author: { type: 'string', required: true },
    category: { type: 'string', required: true },
    tier: { type: 'enum', options: ['public', 'member', 'inner_circle', 'client'], required: true },
    tags: { type: 'list', of: { type: 'string' } },
    featured: { type: 'boolean', default: false },
    draft: { type: 'boolean', default: false },
    image: { type: 'string' },
  },
  computedFields: {
    slug: { type: 'string', resolve: (doc) => doc._raw.flattenedPath.replace('briefs/', '') },
    url: { type: 'string', resolve: (doc) => `/briefs/${doc.slug}` },
    readingTime: { type: 'string', resolve: (doc) => calculateReadingTime(doc.body.raw) },
  },
}));
```

Additional document types: `BlogPost`, `VaultEntry`, `RegistryItem`, `CaseStudy`, `Framework`, `Lexicon`, `Toolkit`, `Playbook`, `Template`, `Assessment`, `Testimonial`, and others.

Frontmatter Schema

Every MDX file must include valid frontmatter. Universal required fields:

Field	Type	Required	Description
`title`	string	Yes	Display title (max 70 chars for SEO)
`description`	string	Yes	Meta description (max 160 chars)
`date`	ISO date	Yes	Publication date
`author`	string	Yes	Author identifier
`draft`	boolean	No	If true, excluded from production builds

Type-specific required fields:

Document Type	Additional Required Fields
Brief	`category`, `tier`
BlogPost	`category`
VaultEntry	`tier`, `access_requirement`
CaseStudy	`client_type`, `outcome`, `duration`
Framework	`version`, `domain`
Lexicon	`term`, `definition_short`

Processing Pipeline

Content flows through a six-stage pipeline:

```
1. PARSE      → Read MDX file, extract frontmatter (gray-matter)
2. VALIDATE   → Check frontmatter against document type schema
3. COMPUTE    → Generate computed fields (slug, URL, reading time, TOC)
4. TRANSFORM  → Apply remark/rehype plugins to MDX body
5. SANITIZE   → Strip potentially dangerous HTML (DOMPurify)
6. OUTPUT     → Write typed JSON to .contentlayer/generated/
```

MDX Processing

MDX files use remark-gfm for GitHub Flavored Markdown and rehype-slug + rehype-autolink-headings for navigation. The build pipeline includes:

- **MDX integrity check** (`scripts/mdx-integrity-check.mjs`): Validates that upstream scripts have not stripped or escaped MDX component tags
- **MDX gate** (`scripts/mdx-illegal-jsx-gate.mjs`): Blocks builds if illegal JSX patterns are detected

Both checks run as pre-commit hooks and during the build pipeline.

Remark Plugins

Plugin	Purpose
`remark-gfm`	GitHub Flavored Markdown (tables, strikethrough, task lists, autolinks)
`remark-reading-time`	Computes estimated reading time from word count
`remark-toc`	Generates table of contents from headings
`remark-unwrap-images`	Removes wrapping `<p>` tags from images
`remark-breaks`	Converts single newlines to ` `

Rehype Plugins

Plugin	Purpose
`rehype-slug`	Adds `id` attributes to headings (enables anchor links)
`rehype-autolink-headings`	Wraps heading text in clickable anchor
`rehype-pretty-code`	Syntax highlighting via Shiki

Plugin order matters — `rehype-slug` must run before `rehype-autolink-headings`.

Generated Types Usage

```
import { allBriefs, type Brief } from 'contentlayer/generated';

export function getPublishedBriefs(): Brief[] {
  return allBriefs
    .filter((brief) => !brief.draft)
    .sort((a, b) => new Date(b.date).getTime() - new Date(a.date).getTime());
}

export function getBriefBySlug(slug: string): Brief | undefined {
  return allBriefs.find((brief) => brief.slug === slug);
}
```

Content Routing

Content Type	Source Directory	Public Route	Access Control
Briefs	`content/briefs/`	`/briefs/[slug]`	Tier-based (per document)
Blog	`content/blog/`	`/blog/[slug]`	Public
Vault	`content/vault/`	`/vault/[slug]`	INNER_CIRCLE minimum
Registry	`content/registry/`	`/registry/[slug]`	MEMBER minimum
Lexicon	`content/lexicon/`	`/lexicon/[slug]`	Public
Frameworks	`content/frameworks/`	`/frameworks/[slug]`	Tier-based

ISR and Cache Strategy

Content Type	Revalidation Period	Rationale
Blog posts	3600s (1 hour)	Updated occasionally, freshness not critical
Briefs	3600s (1 hour)	Same as blog
Vault content	3600s (1 hour)	Gated content, low update frequency
Homepage	1800s (30 min)	Features rotating content
Pricing page	86400s (24 hours)	Rarely changes
Legal pages	`force-static`	Effectively never changes

Content pages use `generateStaticParams()` for static generation at build time with ISR for updates:

```
// pages/briefs/[slug].tsx (or equivalent App Router route)
import { allBriefs } from 'contentlayer/generated';

export async function generateStaticParams() {
  return allBriefs
    .filter((brief) => !brief.draft)
    .map((brief) => ({ slug: brief.slug }));
}

export const revalidate = 3600; // ISR: regenerate hourly
```

Canon Codes

Canon entries use a structured code system for referencing decision authority principles. The `CanonEntry` Prisma model stores canonical references with their codes, titles, and relationships.

Glossary

The glossary system (`scripts/glossary-injector.mjs`) injects term definitions into content at build time, ensuring consistent terminology across all published material.

Toolkits

Published toolkits contain conceptual models only. No numeric frameworks, scoring brackets, or axis names appear in public-facing toolkit content. This is an IP protection requirement.

Windows Compatibility

Contentlayer2 has known issues on Windows due to file path handling. The platform includes several compatibility layers:

1. `contentlayer.windows-fix.mjs` — Patches path resolution to use forward slashes
2. `contentlayer.windows.config.ts` — Windows-specific configuration overrides
3. `scripts/contentlayer-windows-fix.js` — Pre-build script that normalizes paths
4. `lib/contentlayer-generated.ts` — Compatibility import wrapper

```
# Development on Windows
pnpm contentlayer:fix   # Runs scripts/contentlayer-windows-fix.js
pnpm contentlayer build # Then build normally
```

> **WARNING**
>
> CI/CD runs on Linux. If content builds succeed locally on Windows but fail in CI, the issue is almost always path separators (`\` vs `/`). Always use the compatibility wrapper for imports and test content builds in CI before merging.

PART III — DOMAIN SYSTEMS

Chapter 7: Diagnostic Engine

The diagnostic engine is the core domain system. It processes user-submitted decision situations through a multi-stage pipeline.

Architecture Overview

The diagnostic engine consists of over 50 source files in `lib/diagnostics/` and `lib/decision/` . Data flows through a strict pipeline:

```
Input → runtime-validation → decision-engine → evidence-graph → narrative-engine → Output
                                        ↓
                                cost-of-delay-engine
                                pattern-recurrence
                                predictive-consequence
                                layered-intelligence-engine
                                cross-respondent-engine
```

> **KEY PRINCIPLE** The diagnostic pipeline is pure — each module receives typed input and produces typed output. No module may read from the database directly. All data access happens at the boundary (API route handlers) and is passed in as arguments.

Core Modules

Module	Path	Purpose
`decision-engine.ts`	`lib/diagnostics/decision-engine.ts`	Scores raw responses against weighted criteria, producing numeric severity and categorical bands
`runtime-validation.ts`	`lib/diagnostics/runtime-validation.ts`	Validates all input against Zod schemas before any processing begins
`cross-respondent-engine.ts`	`lib/diagnostics/cross-respondent-engine.ts`	Merges and reconciles responses from multiple respondents into a unified assessment
`evidence-graph.ts`	`lib/diagnostics/evidence-graph.ts`	Links scored indicators to source evidence, building a directed graph of causal relationships
`cost-of-delay-engine.ts`	`lib/diagnostics/cost-of-delay-engine.ts`	Calculates ROI impact of inaction
`layered-intelligence-engine.ts`	`lib/diagnostics/layered-intelligence-engine.ts`	Multi-level analysis combining surface indicators, structural patterns, and deep signals
`narrative-engine.ts`	`lib/diagnostics/narrative-engine.ts`	Transforms structured analytical output into natural-language prose
`pattern-recurrence.ts`	`lib/diagnostics/pattern-recurrence.ts`	Detects recurring behavioural and structural patterns across time
`predictive-consequence.ts`	`lib/diagnostics/predictive-consequence.ts`	Projects future states based on current trajectory

CaseObject

Defined in `lib/decision/case-object.ts`. The CaseObject captures 6 user inputs preserved verbatim:

1. `decision` — the decision in the user's own words

2. `priorAttempt` — what they already tried
3. `costOfDelay` — what gets more expensive each week
4. `claimedOwner` — who they say owns the decision
5. `blocker` — what they say is blocking the decision
6. `forcedAction` — what they would do if forced to decide in 24 hours

The CaseObject also carries derived fields: `contradiction`, `inferredAvoidance`, `conditionClass`, `signalStrength`, and `specificityScore`. These are computed server-side, never client-supplied.

C3 Fidelity Scorer

Defined in `lib/decision/c3-fidelity-scorer.ts`. Scores three dimensions:

- **Clarity** (0-1): Is the decision itself clear?
- **Context** (0-1): Is there enough surrounding information?
- **Consequence** (0-1): Is the cost/stakes articulated?

Tiered enforcement based on the composite specificity score:

Tier	Score Range	Behavior
HARD_RECOVERY	< 0.5	No synthesis. Recovery questions only.
SOFT_RECOVERY	0.5 - 0.7	Deterministic output only. No contradiction block.
FULL_SYNTHESIS	>= 0.7	Full LLM synthesis + arbiter tournament.

Synthesis Engine

Defined in `lib/decision/synthesis-engine.ts`. Imports `server-only` — cannot be bundled into the client.

The engine produces a `GovernedSynthesis` output containing: verdict, primary contradiction, avoided decision, why prior attempts failed, concrete move, default path forecast, signal strength, certainty boundary, and quoted user language.

Architecture:

1. CaseObject -> C3 Score -> tier check
2. HARD_RECOVERY -> recovery questions only
3. SOFT_RECOVERY -> deterministic fallback only
4. FULL_SYNTHESIS -> LLM synthesis -> arbiter tournament -> governed output
5. Arbiter rejection -> explicit mismatch message, NOT silent fallback

Arbiter Tournament

Defined in `lib/decision/arbiter-tournament.ts`. Five mandatory rules — if any hard rule fails, synthesis is rejected:

1. **Condition integrity** — synthesis condition class must match deterministic classification
2. **Contradiction alignment** — must reference terms from the deterministic contradiction set

3. **Move validity** — must reference the stated blocker, not generic advice
4. **Cost consistency** — no invented costs without user-stated cost data
5. **Avoidance proof** — must reference forcedAction or priorAttempt mismatch

Violations are classified as `hard` (reject) or `soft` (warn but allow). The user sees the mismatch when hard violations occur.

Assessment Contracts

Every diagnostic module adheres to a typed contract:

```
// Input contract
interface DiagnosticInput {
  sessionId: string;
  respondentId: string;
  responses: ValidatedResponse[];
  context: AssessmentContext;
  options?: DiagnosticOptions;
}

// Output contract — all modules produce this shape
interface DiagnosticResult {
  scores: ScoredDimension[];
  evidence: EvidenceNode[];
  severity: SeverityLevel;
  confidence: number; // 0-1
  narrative?: string;
  metadata: ResultMetadata;
}
```

Result builders construct the final output:

```
// lib/diagnostics/builders/result-builder.ts
export function buildDiagnosticResult(
  scores: ScoredDimension[],
  evidence: EvidenceNode[],
  options: BuilderOptions
): DiagnosticResult {
  return {
    scores,
    evidence,
    severity: deriveSeverity(scores),
    confidence: calculateConfidence(evidence),
    narrative: options.includeNarrative
      ? generateNarrative(scores, evidence)
      : undefined,
    metadata: {
      generatedAt: new Date().toISOString(),
      engineVersion: ENGINE_VERSION,
      modulesUsed: options.modules,
    },
  };
}
```

> **WARNING** Never construct a `DiagnosticResult` manually. Always use the result builder — it enforces invariants.

Severity

The scoring system in `lib/diagnostics/scoring.ts` uses 4 severity tiers:

Severity	Score Range
systemic	< 15
critical	15 - 34
high	35 - 54
moderate	55 - 74
low	75 - 89
negligible	>= 90

IMPLEMENTED: The full 6-tier severity model is live in `scoring.ts`, `client.ts`, `store.ts`, and `types.ts`:

```
// Target severity levels — ordered by urgency
enum SeverityLevel {
  NEGLIGIBLE = 'NEGLIGIBLE',
  LOW = 'LOW',
  MODERATE = 'MODERATE',
  HIGH = 'HIGH',
  CRITICAL = 'CRITICAL',
  SYSTEMIC = 'SYSTEMIC',
}
```

Condition Classes

Four condition classes, defined in `lib/decision/case-object.ts`:

- `authority` — ownership and mandate failures
- `definition` — the decision itself is unclear
- `execution` — the decision is clear but execution stalls
- `instability` — structural instability across multiple dimensions

Contradiction Archetypes

Currently implemented in `lib/diagnostics/contradictions.ts` — 3 core archetypes with variant expressions:

1. **URGENCY_VS_OWNERSHIP** — high urgency + unclear ownership
2. **CLARITY_VS_ACCOUNTABILITY** — defined decision + no accountability
3. **URGENCY_VS_STATE** — urgent + repeatedly deferred

IMPLEMENTED: The full 13-archetype model is live in `lib/diagnostics/signals.ts`. The `inferArchetypeSignal()` function in `lib/decision/synthesis-engine.ts` selects the specific variant from user input. Authority class: AUTHORITY_LEAKAGE, AUTHORITY_CONTEST, AUTHORITY_VACUUM, FALSE_AUTHORITY. Definition class: DEFINITION_FAILURE, DEFINITION_DRIFT, DEFINITION_CONFLICT. Execution class: EXECUTION_AVOIDANCE, EXECUTION_THEATRE, ESCALATION_AVOIDANCE. Instability class: LATENT_INSTABILITY, STRUCTURAL_FRAGILITY, GOVERNANCE_EROSION.

Server-Side Scoring

ALL scoring runs server-side via `/api/diagnostics/score`. The client is submit-only:

1. Collects answers from the user
2. POSTs to the scoring API
3. Renders the public DTO that comes back

No thresholds, weights, formulas, or classification rules exist in the client bundle.

Diagnostic API Routes

Method	Route	Purpose
`POST`	`/api/diagnostics/score`	Server-side fast diagnostic scoring
`POST`	`/api/diagnostics/submit`	Submit diagnostic for persistence
`GET`	`/api/diagnostics/[ref]`	Retrieve diagnostic by reference
`POST`	`/api/diagnostics/sessions/[id]/responses`	Submit responses
`POST`	`/api/diagnostics/sessions/[id]/generate`	Trigger report generation
`GET`	`/api/diagnostics/sessions/[id]/report`	Retrieve generated report
`GET`	`/api/diagnostics/sessions/[id]/artifacts`	List artifacts for session

All routes follow the standard response format:

```
// Success
{ ok: true, data: { ... }, code: 200 }

// Error
{ ok: false, error: "Descriptive message", code: 422 }
```

Diagnostic PDF Generation

Diagnostic reports generate PDFs through the unified pipeline (see Chapter 36). The diagnostic-specific flow:

```
DiagnosticResult → template selection → data hydration → React PDF render → storage
```

Templates are selected based on `DiagnosticType` and `ReportFormat`:

```
export function selectTemplate(
  type: DiagnosticType,
  format: ReportFormat
): PDFTemplate {
  const key = `${type}:${format}`;
  const template = TEMPLATE_REGISTRY.get(key);
  if (!template) {
    throw new DiagnosticError(`No template registered for ${key}`);
  }
  return template;
}
```

Storage Adapter

Diagnostic artifacts and generated PDFs use a configurable storage backend:

```
// lib/diagnostics/storage.ts
interface StorageAdapter {
  put(key: string, data: Buffer, options?: StorageOptions): Promise<string>;
  get(key: string): Promise<Buffer | null>;
  delete(key: string): Promise<void>;
  getSignedUrl(key: string, expiresIn?: number): Promise<string>;
}
```

Chapter 8: Alignment & Campaign System

Purpose Alignment

The Purpose Alignment assessment (`app/purpose-alignment/`) evaluates individual alignment across six domains defined in the `AlignmentDomain` enum:

- IDENTITY
- DECISION
- ENVIRONMENT
- BEHAVIOUR
- EMOTIONAL_ORDER
- LEGACY

Results are classified into four bands (`AlignmentBand`): ALIGNED, DRIFTING, MISALIGNED, DISORDERED.

Evidence persistence and consumption: PA results are persisted to the DiagnosticJourney store via `persistDiagnosticStage()` in the assessments API route (`app/api/purpose-alignment/assessments/route.ts`). The evidence is retrievable via `loadPurposeAlignmentEvidence()` in `lib/alignment/evidence-loader.ts` and rendered on all downstream surfaces (Executive Reporting, Strategy Room, Return Brief, Decision Centre, Oversight Brief) via the `GovernanceEvidenceCarryForward` component with source labels like `"CAPTURED in Purpose Alignment"` and `"Previously reported competing obligation"` .

AlignmentBand Classification

```
export function classifyBand(score: number): AlignmentBand {
  if (score >= 75) return AlignmentBand.ALIGNED;
  if (score >= 50) return AlignmentBand.DRIFTING;
  if (score >= 25) return AlignmentBand.MISALIGNED;
  return AlignmentBand.DISORDERED;
}
```

> **KEY PRINCIPLE** Band thresholds are institutional constants. They do not change per client, per campaign, or per domain.

Organisation Model

```
model Organisation {
  id          String   @id @default(cuid())
  name        String
  slug        String   @unique
  sector      String?
  sizeBand    String?
  region      String?
  status      String   @default("active")
  logoUrl     String?
  domain      String?  @unique
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  memberships OrganisationMembership[]
  campaigns   AlignmentCampaign[]

  @@index([slug])
  @@index([domain])
}

model OrganisationMembership {
  id             String   @id @default(cuid())
  organisationId String
  email          String
  roleTitle      String?
  seniorityBand  String?
  isExecutive    Boolean  @default(false)
  status         String   @default("active")
  joinedAt       DateTime @default(now())
  organisation   Organisation @relation(fields: [organisationId], references: [id])

  @@unique([organisationId, email])
}
```

AlignmentCampaign Model

```
model AlignmentCampaign {
  id              String   @id @default(cuid())
  organisationId  String
  title           String
  description     String?
  diagnosticType  String
  stage           String   @default("draft")
  cadence         String?
  deadline        DateTime?
  createdById     String
  createdAt       DateTime @default(now())
  closedAt        DateTime?
  organisation    Organisation @relation(fields: [organisationId], references: [id])
  participants    CampaignParticipant[]

  @@index([organisationId, stage])
}

model CampaignParticipant {
  id          String   @id @default(cuid())
  campaignId  String
  email       String
  userId      String?
  respondentType String?
  status      String   @default("invited")
  invitedAt   DateTime @default(now())
  respondedAt DateTime?
  campaign    AlignmentCampaign @relation(fields: [campaignId], references: [id])

  @@unique([campaignId, email])
}
```

Campaign Lifecycle

```
CREATE → INVITE → RESPOND → CLOSE → REPORT
  ↓        ↓        ↓         ↓        ↓
Draft    Emails   Collect   Lock     Generate
campaign  sent    responses  input    analysis
```

Each transition is gated:

Transition	Gate
CREATE → INVITE	Campaign must have at least one participant added
INVITE → RESPOND	At least one invite accepted (tracked via `CampaignParticipant.status`)
RESPOND → CLOSE	Owner explicitly closes OR deadline reached
CLOSE → REPORT	All responses validated, cross-respondent merge complete

> **WARNING** Once a campaign moves to CLOSE, no further responses are accepted. This is irreversible.

Enterprise Campaigns

Enterprise campaigns (`AlignmentCampaign` model) support organisation-wide diagnostic deployment. A campaign has:

- An organisation owner
- A lifecycle (draft -> intake -> active -> closed -> archived)
- Participants invited via `CampaignParticipant` with token-based access
- Team and organisation-level assessment snapshots
- Executive reporting runs that produce governance artifacts

Scoring Domains

Domain	Code	Description
Identity	`IDENTITY`	Clarity of self-concept, values articulation, role congruence
Decision	`DECISION`	Decision-making quality, speed, consistency with stated values
Environment	`ENVIRONMENT`	Physical and relational context, structural enablers/blockers
Behaviour	`BEHAVIOUR`	Observable actions vs. stated intentions, habit architecture
Emotional Order	`EMOTIONAL_ORDER`	Emotional regulation, response patterns, resilience indicators
Legacy	`LEGACY`	Long-term orientation, generational thinking, impact trajectory

Enterprise Assessment Generation

```
export async function generateEnterpriseAssessment(
  organisationId: string,
  options: EnterpriseAssessmentOptions
): Promise<EnterpriseAssessment> {
  const campaigns = await getCampaignsForOrg(organisationId, {
    status: 'CLOSED',
    dateRange: options.dateRange,
  });

  const aggregated = aggregateScores(campaigns);
  const interventions = deriveInterventions(aggregated);
  const narrative = await generateEnterpriseNarrative(aggregated, interventions);

  return {
    organisationId,
    generatedAt: new Date().toISOString(),
    domainScores: aggregated.domainScores,
    overallBand: classifyBand(aggregated.overallScore),
    interventions,
    narrative,
    participantCount: aggregated.totalParticipants,
    campaignCount: campaigns.length,
  };
}
```

Strategic Interventions

```
interface StrategicIntervention {
  id: string;
  domain: ScoringDomain;
  severity: SeverityLevel;
  title: string;
  description: string;
  recommendedActions: string[];
  timeframe: 'IMMEDIATE' | '30_DAYS' | '90_DAYS' | '6_MONTHS';
  estimatedImpact: number; // 0-1
}
```

Band	Timeframe	Action Type
DISORDERED	IMMEDIATE	Crisis stabilisation
MISALIGNED	30_DAYS	Structural correction
DRIFTING	90_DAYS	Course adjustment
ALIGNED	6_MONTHS	Optimisation / maintenance

Campaign API Routes

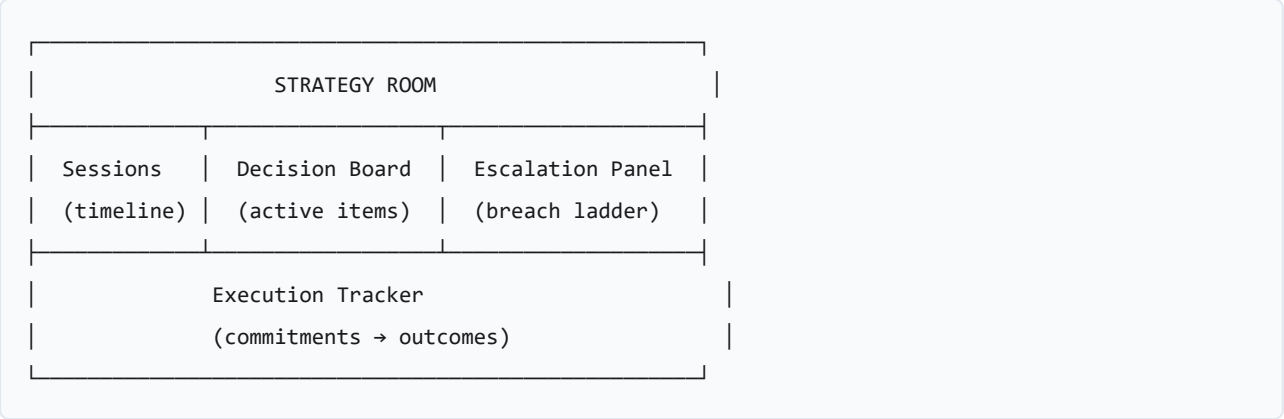
Method	Route	Purpose
`POST`	`/api/alignment/campaigns`	Create campaign
`GET`	`/api/alignment/campaigns`	List campaigns for org
`GET`	`/api/alignment/campaigns/[id]`	Get campaign detail
`PATCH`	`/api/alignment/campaigns/[id]`	Update campaign (status, metadata)
`POST`	`/api/alignment/campaigns/[id]/invite`	Invite participants
`POST`	`/api/alignment/campaigns/[id]/respond`	Submit alignment response
`POST`	`/api/alignment/campaigns/[id]/close`	Close campaign
`GET`	`/api/alignment/campaigns/[id]/report`	Get alignment report
`POST`	`/api/alignment/enterprise/assess`	Generate enterprise assessment
`GET`	`/api/alignment/organisations/[id]`	Get organisation detail
`POST`	`/api/alignment/organisations`	Create organisation
`PATCH`	`/api/alignment/organisations/[id]/members`	Manage membership

OGR (Organisation Governance Report)

Executive reporting runs (`ExecutiveReportingRun`) produce artifacts (`ExecutiveReportingArtifact`) that summarize campaign findings at the board level. Reports are generated as PDFs via the App Router's PDF rendering pipeline.

Chapter 9: Strategy Room & Execution

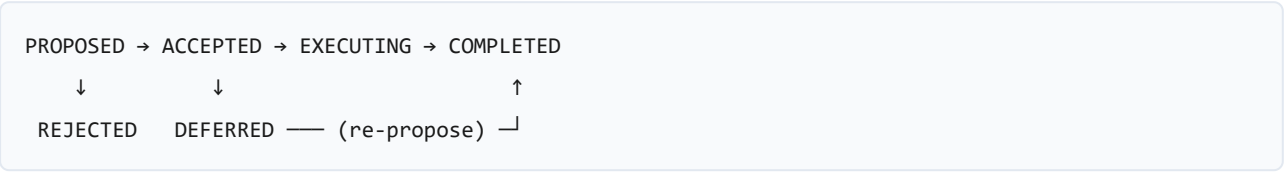
Strategy Room Architecture



The Strategy Room (`app/strategy-room/` , `StrategyRoomSession` model) provides execution infrastructure for decisions that have been diagnosed. It operates through:

1. **Intake:** `StrategyIntake` and `StrategyInquiry` models capture the decision context
2. **Session management:** `StrategyRoomSession` tracks active execution sessions
3. **Execution tracking:** `StrategyRoomExecutionSession` and `StrategyDecisionLog` record actions taken
4. **Recommendation engine:** `StrategyRoomRecommendationImpression` , `StrategyRoomFollowup` , `StrategyRoomConversion` track recommendation effectiveness

Decision Tracking State Machine



Every decision must have:

1. A clear statement (what is being decided)
2. An owner (who is accountable)
3. A deadline (when it must be resolved)
4. A rationale (why this decision, not another)

> **WARNING** Decisions without deadlines are not decisions — they are wishes. The system enforces deadline assignment at creation time.

Breach Ladder Implementation

The breach ladder is a time-based escalation framework for commitments that are not being honoured:

```
enum BreachLevel {
  NONE = 0,       // Commitment on track
  NOTICE = 1,     // 24h past deadline — gentle reminder
  WARNING = 2,    // 48h — formal warning
  ESCALATION = 3, // 72h — escalated to stakeholders
  BREACH = 4,     // 5 days — formal breach recorded
  CRITICAL = 5,   // 7 days — systemic failure flag
}
```

The escalation engine runs on a cron schedule:

```
export async function processEscalations(): Promise<EscalationResult[]> {
  const overdue = await getOverdueCommitments();
  const results: EscalationResult[] = [];

  for (const commitment of overdue) {
    const hoursOverdue = differenceInHours(new Date(), commitment.deadline);
    const newLevel = determineBreachLevel(hoursOverdue);

    if (newLevel > commitment.breachLevel) {
      await escalate(commitment, newLevel);
      results.push({
        commitmentId: commitment.id,
        previousLevel: commitment.breachLevel,
        newLevel,
        action: getEscalationAction(newLevel),
      });
    }
  }

  return results;
}
```

Escalation actions by level:

Level	Hours Overdue	Action
NOTICE	24h	In-app notification to commitment owner
WARNING	48h	Email to owner + session log entry
ESCALATION	72h	Notification to all session stakeholders
BREACH	5 days	Formal breach record, visible in reports
CRITICAL	7 days	Systemic failure flag, blocks new commitments

> **KEY PRINCIPLE** The escalation engine is mechanical and impartial. It does not negotiate, it does not accept excuses, it does not reset without explicit action.

Enforcement

The enforcement system uses:

- **Breach ladder:** Escalating consequences tracked via `EscalationEvent`
- **Enforcement playbooks:** `EnforcementPlaybook` and `PlaybookApplication` models
- **Enforcement cycles:** `EnforcementCycle` model for recurring governance enforcement
- **Pattern breaker contracts:** `PatternBreakerContract` model for formalizing commitment to change

Execution Tracking

- `DecisionJourneyEvent` records decision lifecycle events
- `ConsequenceTimeline` tracks projected and actual consequences
- `CalibrationState` and `CalibrationEvent` track calibration of the scoring system against real outcomes
- `OutcomeVerificationRecord` stores verified decision outcomes for longitudinal analysis

---

Chapter 10: Commercial Layer

Stripe Integration

Stripe handles payment processing for diagnostic reports and premium access:

```
// lib/stripe/client.ts
import Stripe from 'stripe';

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2024-04-10',
  typescript: true,
});

// Webhook signature verification — ALWAYS verify
export function verifyWebhookSignature(
  payload: string | Buffer,
  signature: string
): Stripe.Event {
  return stripe.webhooks.constructEvent(
    payload,
    signature,
    process.env.STRIPE_WEBHOOK_SECRET!
  );
}
```

> **WARNING** Never trust client-side payment confirmations. All payment state changes must be driven by verified Stripe webhooks.

Key flows:

- **Checkout:** `pages/api/diagnostics/create-report-checkout.ts` creates Stripe checkout sessions for diagnostic report purchases
- **Webhooks:** `pages/api/webhooks/` processes Stripe events (payment success, subscription changes)
- **Entitlements:** `Entitlement` model grants access after successful payment. `ClientEntitlement` provides client-tier access.

Webhook events handled:

```
const HANDLED_EVENTS = [
  'checkout.session.completed',
  'customer.subscription.created',
  'customer.subscription.updated',
  'customer.subscription.deleted',
  'invoice.payment_succeeded',
  'invoice.payment_failed',
] as const;
```

Pricing Engine

```
interface PricingTier {
  id: string;
  name: string;
  stripePriceId: string;
  features: Feature[];
  limits: TierLimits;
  amount: number;        // in smallest currency unit (pence)
  currency: string;      // ISO 4217
  interval: 'month' | 'year' | 'once';
}

interface TierLimits {
  diagnosticSessions: number;    // -1 = unlimited
  campaignParticipants: number;
  storageBytes: number;
  pdfGenerations: number;
  apiCalls: number;
}
```

Checkout Flow

```
Client → /api/checkout/create-session → Stripe Checkout → Webhook → Entitlement
```

Entitlement Verification

The `Entitlement` model supports three types (`EntitlementType`): TIER, PRODUCT, ARTIFACT. Each entitlement has a status (ACTIVE, REVOKED, EXPIRED) with start/expiry dates and audit trail (issuedBy, revokedBy, reason).

```
export async function hasEntitlement(
  userId: string,
  type: EntitlementType,
  reference?: string
): Promise<boolean> {
  const entitlement = await db.entitlement.findFirst({
    where: {
      userId,
      type,
      ...(reference ? { reference } : {}),
      revoked: false,
      OR: [
        { expiresAt: null },
        { expiresAt: { gt: new Date() } },
      ],
    },
  });
  return !!entitlement;
}
```

Report Purchase Flow

1. User completes diagnostic -> receives `diagnosticId` and `diagnosticRef`
2. User initiates report purchase -> system creates `DiagnosticReportOrder` + Stripe checkout
3. Payment confirmed -> `reportStatus` updated on `DiagnosticRecord`
4. Report generated -> `DiagnosticArtifact` created with PDF storage reference
5. User downloads via access-controlled route

Access Key Management

Access keys provide API-level authentication for programmatic access:

```
export async function createAccessKey(
  userId: string,
  options: AccessKeyOptions
): Promise<{ key: string; id: string }> {
  const rawKey = generateSecureKey(); // crypto.randomBytes(32)
  const hashedKey = await hashKey(rawKey);

  const record = await db.accessKey.create({
    data: {
      userId,
      hashedKey,
      name: options.name,
      scopes: options.scopes,
      expiresAt: options.expiresAt,
    },
  });

  // Return raw key ONCE — it is never stored or retrievable
  return { key: `aol_${rawKey}`, id: record.id };
}
```

> **WARNING** Access keys are shown exactly once at creation time. They are stored as Argon2 hashes. If a user loses their key, it must be revoked and a new one issued.

Inner Circle Membership

```
interface InnerCircleMembership {
  userId: string;
  status: 'ACTIVE' | 'PAUSED' | 'CANCELLED' | 'LAPSED';
  tier: 'FOUNDING' | 'STANDARD';
  stripeSubscriptionId: string;
  startedAt: string;
  currentPeriodEnd: string;
  benefits: InnerCircleBenefit[];
}

enum InnerCircleBenefit {
  UNLIMITED_DIAGNOSTICS = 'UNLIMITED_DIAGNOSTICS',
  PRIORITY_SUPPORT = 'PRIORITY_SUPPORT',
  EARLY_ACCESS = 'EARLY_ACCESS',
  EXCLUSIVE_CONTENT = 'EXCLUSIVE_CONTENT',
  STRATEGY_SESSIONS = 'STRATEGY_SESSIONS',
  ENTERPRISE_FEATURES = 'ENTERPRISE_FEATURES',
}
```

PART IV — PRODUCT ELEVATION LAYER

Chapter 11: Cost of Inaction Engine

File: `lib/server/decision/cost-of-inaction.server.ts`

Server-only module (`import "server-only"`). Produces qualitative exposure assessments, not numeric calculations.

Input

```
type CostOfInactionInput = {
  state: "ORDERED" | "DRIFTING" | "MISALIGNED" | "DISORDERED";
  estimatedExposureGBP?: number | null;
  decisionWindow?: string | null;
  headcountAffected?: number | null;
  marketExposure?: string | null;
};
```

Output

```
type CostOfInactionPublic = {
  exposureBand: "low" | "moderate" | "high" | "critical" | "undisclosed";
  horizon30: string;
  horizon60: string;
  horizon90: string;
  executiveWarning: string;
};
```

The engine classifies exposure into bands based on state and financial anchors when available. It produces 30/60/90-day horizon narratives describing projected deterioration in natural language. No formulas or multipliers are exposed in the public output.

When financial data is not provided, the engine derives the band from state alone (DISORDERED -> critical, MISALIGNED -> high, DRIFTING -> moderate). The `undisclosed` band is used when no state or financial anchor is sufficient for classification.

Persistence: The computed exposure band and horizon narratives are persisted to the DiagnosticJourney store via `persistFinancialExposureSnapshot()` and `persistCostOfInactionProjection()` in `lib/product/financial-exposure-persistence.ts`. This ensures downstream surfaces (Return Brief, Decision Centre, Oversight Brief) can retrieve what the system originally told the user about delay, consequence, and inaction.

Chapter 12: Execution Failure Predictor

File: `lib/server/decision/execution-failure.server.ts`

Server-only module. Performs state-based assessment of likely execution failure modes.

Input

```
type ExecutionFailureInput = {
  state: string;
  publicConditions?: string[];
  directive: string;
};
```

Output

```
type ExecutionFailurePublic = {
  likelyFailureMode: string;
  whyExecutionWillStall: string;
  requiredCorrection: string;
};
```

The engine pattern-matches on public-safe state and conditions to predict the specific failure mode. For example:

- DISORDERED -> "Authority collapse" — no single authority can enforce the directive
- MISALIGNED + authority conditions -> ownership ambiguity failures
- DRIFTING -> passive consensus seeking that produces no binding outcome

Chapter 13: Decision Authority Index

File: `lib/server/decision/authority-index.server.ts`

Server-only module. Produces an interpretive governance band, not a numeric score.

Output

```
type DecisionAuthorityIndexPublic = {
  band: "strong" | "strained" | "weak" | "critical";
  label: string;
  boardMeaning: string;
  nextGovernanceMove: string;
};
```

The index classifies decision authority into four bands:

- **critical** — decision authority has failed; board-level intervention indicated
- **weak** — governance structure is producing decisions but cannot enforce them
- **strained** — functional but showing signs of degradation
- **strong** — governance structure is producing and enforcing decisions

Chapter 14: Decision Memory Service

File: `lib/server/decision-memory/memory-service.server.ts`

Server-only module. CRUD operations plus trend analysis backed by the `DecisionMemory` Prisma model.

Operations

- `createDecisionMemory(input)` — persists a new decision memory record
- `listDecisionMemoryByUser(userId)` — retrieves up to 50 most recent records
- `summariseDecisionMemoryTrend(userId)` — computes trend analysis

Trend Analysis

```
type DecisionMemoryTrend = {
  totalDecisions: number;
  dominantState: string;
  repeatedConditions: string[];
  escalationTrend: "stable" | "rising" | "falling" | "insufficient_data";
  executiveSummary: string;
};
```

PART V — PRESENTATION LAYER

Chapter 15: Styling Architecture

Design System: Institutional Monumentalism

The visual language is dark, authoritative, and precise. It communicates institutional weight, not startup energy.

Design Tokens

Token	Value
Gold (primary accent)	`#C9A96E` / `rgb(201 169 110)`
Gold Strong	`rgb(212 175 55)`
Surface (background)	`rgb(14 14 18)`
Surface 2	`rgb(9 9 12)`
Surface 3	`rgb(6 6 9)`

Typography

Role	Font
Serif (headings, institutional)	Cormorant Garamond
Sans (body, UI)	Inter (via `var(--font-sans)`)
Mono (code, data)	JetBrains Mono

Tailwind Configuration

Configured in `tailwind.config.cjs`. Dark mode uses the `class` strategy. The theme extends the default with:

- Custom color scales (gold, surface, ds.* design system tokens)
- Gold gradient backgrounds (`gold-radial`, `gold-linear`)
- Panel shadows with gold accents (`aol-panel-gold`)
- Custom animations (`pulse-gold`, `fade-in`)
- Container with centered layout, max-width 1440px

```
// tailwind.config.cjs (key excerpts)
module.exports = {
  darkMode: 'class',
  content: [
    './app/**/*.{ts,tsx,mdx}',
    './pages/**/*.{ts,tsx}',
    './components/**/*.{ts,tsx}',
    './lib/**/*.{ts,tsx}',
    './content/**/*.mdx',
  ],
  theme: {
    extend: {
      colors: {
        aol: {
          bg: 'var(--aol-bg)',
          ink: 'var(--aol-ink)',
          gold: 'var(--aol-gold)',
          muted: 'var(--aol-muted)',
          border: 'var(--aol-border)',
          surface: 'var(--aol-surface)',
          accent: 'var(--aol-accent)',
        },
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
        serif: ['Cormorant Garamond', 'Georgia', 'serif'],
        mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
      },
    },
  },
};
```

Dark Mode CSS Variables

```
:root {
  --aol-bg: #fafaf8;
  --aol-ink: #1a1a1a;
  --aol-gold: #b8860b;
  --aol-muted: #6b7280;
  --aol-border: #e5e5e0;
  --aol-surface: #ffffff;
  --aol-accent: #1e3a5f;
}

.dark {
  --aol-bg: #0f0f0f;
  --aol-ink: #e8e8e8;
  --aol-gold: #d4a017;
  --aol-muted: #9ca3af;
  --aol-border: #2a2a2a;
  --aol-surface: #1a1a1a;
  --aol-accent: #4a90d9;
}
```

> **KEY PRINCIPLE** Never use raw colour values in components. Always reference the semantic CSS variables via Tailwind classes (`bg-aol-bg` , `text-aol-ink` , `border-aol-border`).

CSS Variables Reference

```
/* Core palette */
--aol-bg           /* Page background */
--aol-ink          /* Primary text */
--aol-gold         /* Accent / brand gold */
--aol-muted        /* Secondary text */
--aol-border       /* Borders and dividers */
--aol-surface      /* Card / elevated surface */
--aol-accent       /* Secondary accent (blue family) */

/* Extended palette */
--aol-success      /* Green — positive states */
--aol-warning      /* Amber — caution states */
--aol-error        /* Red — error / destructive */
--aol-info         /* Blue — informational */

/* Shadows */
--aol-shadow-sm    /* Subtle elevation */
--aol-shadow-md    /* Card elevation */
--aol-shadow-lg    /* Modal / dropdown elevation */
```

Component Variants (CVA Pattern)

```
// components/ui/button.tsx
import { cva, type VariantProps } from 'class-variance-authority';

const buttonVariants = cva(
  'inline-flex items-center justify-center rounded-md font-medium transition-colors ' +
  'focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-aol-gold ' +
  'disabled:pointer-events-none disabled:opacity-50',
  {
    variants: {
      variant: {
        default: 'bg-aol-ink text-aol-bg hover:bg-aol-ink/90',
        destructive: 'bg-red-600 text-white hover:bg-red-700',
        outline: 'border border-aol-border bg-transparent hover:bg-aol-surface',
        ghost: 'hover:bg-aol-surface hover:text-aol-ink',
        link: 'text-aol-gold underline-offset-4 hover:underline',
        institutional: 'bg-aol-gold text-white font-serif tracking-institutional',
      },
      size: {
        sm: 'h-8 px-3 text-xs',
        default: 'h-10 px-4 py-2 text-sm',
        lg: 'h-12 px-6 text-base',
        icon: 'h-10 w-10',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'default',
    },
  }
);
```

Radix UI Integration

Components are composed from Radix primitives, styled with Tailwind + CVA:

```
// Pattern: Radix Primitive → Styled Wrapper → Feature Component
import * as DialogPrimitive from '@radix-ui/react-dialog';

const DialogOverlay = React.forwardRef<...>(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      'fixed inset-0 z-50 bg-black/50 backdrop-blur-sm',
      'data-[state=open]:animate-in data-[state=closed]:animate-out',
      className
    )}
    {...props}
  />
));
```

Responsive Patterns

Breakpoints follow Tailwind defaults with institutional additions:

```
screens: {
  'sm': '640px',
  'md': '768px',
  'lg': '1024px',
  'xl': '1280px',
  '2xl': '1536px',
  'institutional': '1440px', // Design system target width
}
```

Chapter 16: Component Architecture

Component Organization

```
components/
├── ui/              ← Base primitives (Button, Input, Dialog, etc.)
├── layout/          ← Shell, Header, Footer, Sidebar, Navigation
├── features/        ← Domain-specific (DiagnosticCard, CampaignList)
├── forms/           ← Form compositions (ContactForm, AssessmentForm)
├── charts/          ← Data visualization components
├── pdf/             ← PDF-specific React components
└── shared/          ← Cross-cutting (ErrorBoundary, Loading, Empty states)
```

> **KEY PRINCIPLE** The `ui/` directory contains ONLY presentational components with no business logic. Feature components compose UI primitives and connect them to domain state.

Unified Conversion Surface

The diagnostic result page assembles a conversion surface from modular components:

- **CaseActiveBanner** — displays the user's case reference and condition classification
- **ConsequenceTimeline** — visualizes the 7/30/90-day forecast with progressive deterioration
- **LimitationsBlock** — states what the system cannot conclude (certainty boundary)
- **DirectiveCTA** — the primary call to action based on the diagnostic finding
- **FeedbackLoop** — captures user response to the diagnostic output

Animation Patterns (Framer Motion)

```
// Fade in on mount
<motion.div
  initial={{ opacity: 0, y: 8 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.3, ease: 'easeOut' }}
>

// Staggered list
<motion.ul>
  {items.map((item, i) => (
    <motion.li
      key={item.id}
      initial={{ opacity: 0, x: -12 }}
      animate={{ opacity: 1, x: 0 }}
      transition={{ delay: i * 0.05 }}
    />
  ))}
</motion.ul>
```

Icons (Lucide React)

```
import { ChevronRight, AlertTriangle, CheckCircle } from 'lucide-react';

<ChevronRight className="h-4 w-4" />          // Inline / small
<AlertTriangle className="h-5 w-5" />          // Default
<CheckCircle className="h-6 w-6" />            // Prominent
```

> **WARNING** Do not use multiple icon libraries. Lucide React is the sole icon source.

Forms

Forms use controlled components with React state and Zod validation:

```
import { z } from 'zod';

const schema = z.object({
  email: z.string().email('Invalid email address'),
  message: z.string().min(10, 'At least 10 characters'),
});

type FormValues = z.infer<typeof schema>;
```

Tables (@tanstack/react-table)

```
import {
  useReactTable,
  getCoreRowModel,
  getSortedRowModel,
  getPaginationRowModel,
} from '@tanstack/react-table';
```

Charts (Recharts)

```
import { ResponsiveContainer, BarChart, Bar, XAxis, YAxis, Tooltip } from 'recharts';

<ResponsiveContainer width="100%" height={300}>
  <BarChart data={domainScores}>
    <XAxis dataKey="domain" />
    <YAxis domain={[0, 100]} />
    <Tooltip />
    <Bar dataKey="score" fill="var(--aol-gold)" radius={[4, 4, 0, 0]} />
  </BarChart>
</ResponsiveContainer>
```

Toasts (Sonner)

```
import { toast } from 'sonner';

toast.success('Report generated successfully');
toast.error('Failed to save changes');
toast.loading('Generating PDF...');
```

Command Palette (cmdk)

```
import { Command } from 'cmdk';

<Command>
  <Command.Input placeholder="Search commands..." />
  <Command.List>
    <Command.Group heading="Navigation">
      <Command.Item onSelect={() => router.push('/dashboard')}>
        Dashboard
      </Command.Item>
    </Command.Group>
  </Command.List>
</Command>
```

Component Principles

- Components receive public DTOs only, never raw scoring data
- No component computes scores, classifications, or thresholds
- Interactive state uses React hooks (useState, useEffect, useReducer)
- No global state management library (no Redux, no Zustand)

Chapter 17: State Management

Client State

React built-in state only. Components use `useState` and `useEffect` for local state. Form state uses controlled components with React state.

Session Persistence

`sessionStorage` stores the public DTO after diagnostic completion. For the intelligence spine (which contains server-side data), the stored version is encrypted with AES-256-GCM using a per-session key generated via the Web Crypto API. This prevents inspection of spine data through browser dev tools.

No Global Store

There is no Redux, Zustand, Jotai, or other global state management library. State flows through:

1. API response -> component props (server-rendered pages)
2. API response -> React state (client-side fetches)
3. sessionStorage -> React state (page revisits within session)

This simplicity is intentional. The system's complexity lives in the server-side domain layer, not the client state graph.

PART VI — API & SECURITY LAYER

Chapter 18: API Architecture

All API routes live under `pages/api/`. Every route validates input with zod and applies rate limiting.

Route Organization

```
pages/api/
├── auth/           ← NextAuth routes + custom auth endpoints
├── diagnostics/    ← Scoring, submission, reports, artifacts
├── stripe/         ← Stripe checkout + webhooks
├── webhooks/       ← Webhook receivers
├── admin/          ← Admin-only operations
├── inner-circle/   ← Inner circle endpoints
├── strategy-room/  ← Strategy room API
├── billing/        ← Billing endpoints
├── ogr/            ← Organisation governance
├── downloads/      ← Download endpoints
├── contact/        ← Public contact form
├── subscribe/      ← Newsletter
├── health/         ← Health checks
├── cron/           ← Scheduled task endpoints
└── rate-limit/     ← Rate limit status
```

Standard Handler Pattern

Every API route follows this structure:

```
import { NextRequest } from 'next/server';
import { z } from 'zod';

const requestSchema = z.object({
  title: z.string().min(1).max(200),
  description: z.string().optional(),
});

export async function POST(req: NextRequest) {
  // 1. Rate limiting
  const rateLimitResult = await rateLimit(req);
  if (!rateLimitResult.ok) {
    return json({ ok: false, error: 'Rate limit exceeded', code: 429 });
  }

  // 2. Authentication
  const session = await getAuthSession();
  if (!session) return unauthorized();

  // 3. Authorization (role/permission checks)
  if (!hasPermission(session.user, 'create:campaign')) {
    return forbidden();
  }

  // 4. Input validation
  const body = await req.json();
  const parsed = requestSchema.safeParse(body);
  if (!parsed.success) {
    return badRequest(parsed.error.flatten().fieldErrors);
  }

  // 5. Business logic
  try {
    const result = await createCampaign(session.user.id, parsed.data);
    return json({ ok: true, data: result, code: 201 });
  } catch (error) {
    return json({ ok: false, error: 'Failed to create campaign', code: 500 });
  }
}
```

Error Handling (ApiResponse)

Standard response format — every response matches this shape:

```
interface ApiResponse<T = unknown> {
  ok: boolean;
  data?: T;
  error?: string;
  code: number;
}

export function json<T>(response: ApiResponse<T>): NextResponse {
  return NextResponse.json(response, { status: response.code });
}

export function unauthorized() {
  return json({ ok: false, error: 'Authentication required', code: 401 });
}

export function forbidden() {
  return json({ ok: false, error: 'Insufficient permissions', code: 403 });
}

export function badRequest(errors: Record<string, string[]>) {
  return json({ ok: false, error: 'Validation failed', data: errors, code: 422 });
}

export function notFound(resource?: string) {
  return json({ ok: false, error: `${resource ?? 'Resource'} not found`, code: 404 });
}
```

> **WARNING** Never return raw error messages from caught exceptions to the client. Internal errors must be logged server-side and a generic message returned.

CORS Configuration

```
const ALLOWED_ORIGINS = [
  'https://abrahamoflondon.com',
  'https://www.abrahamoflondon.com',
  'https://app.abrahamoflondon.com',
  process.env.NODE_ENV === 'development' && 'http://localhost:3000',
].filter(Boolean) as string[];
```

> **KEY PRINCIPLE** CORS is a whitelist — only explicitly approved origins may call API routes. The wildcard (`*`) is never used in production.

API Versioning

Breaking changes are served under `/api/v2/`:

Versioning rules:

- Additive changes (new fields, new endpoints) — no version bump
- Removing fields, changing types, restructuring — new version
- Old versions are maintained for 6 months after deprecation notice

Core Diagnostic Routes

`POST /api/diagnostics/score`

Server-side Fast Diagnostic scoring. Accepts `{ answers: Record<string, string>, committed: boolean }`. Runs the full scoring pipeline. Returns `FastDiagnosticResult` DTO only.

`POST /api/diagnostics/submit`

Diagnostic submission endpoint. Returns `diagnosticId` and `diagnosticRef` ONLY. No score, severity, or verdict in the response.

`GET /api/diagnostics/constitutional-intake/report`

Returns `PublicConstitutionalResult` only. No raw bundle, scoring data, or engine internals.

Supporting Routes

- `/api/auth/[...nextauth]` — NextAuth authentication
- `/api/auth/identity` — identity resolution
- `/api/auth/session` — session status
- `/api/stripe/` — Stripe checkout and webhook handling
- `/api/diagnostics/create-report-checkout` — report purchase flow
- `/api/rate-limit/` — rate limit status endpoints
- `/api/health` — system health check
- `/api/cron/` — scheduled task endpoints

Chapter 19: Public DTO Contract

FastDiagnosticResult

```
type FastDiagnosticResult = {
  caseRef: string;
  condition: string;
  conditionLabel: string;
  signalStrength: "low" | "moderate" | "high";
  fullAnalysis: boolean;
  recoveryQuestion: string | null;
  synthesis: {
    verdict: string;
    primaryContradiction: string;
    avoidedDecision: string;
    whyPriorAttemptsFailed: string;
    concreteMove: string;
    defaultPathForecast: string;
    certaintyBoundary: string;
    quotedUserLanguage: string[];
  } | null;
  forecast: {
    alreadyIncurred?: string;
    sevenDays: string;
    thirtyDays: string;
    ninetyDays: string;
    optionCompression?: string;
    consequenceShift?: string;
    controlShiftSummary: string;
  } | null;
  contradictionText: string | null;
  arbiterMessage: string | null;
  stateToken: string;
  costOfInaction?: CostOfInactionPublic;
  executionFailure?: ExecutionFailurePublic;
  authorityIndex?: DecisionAuthorityIndexPublic;
  memoryTrend?: DecisionMemoryTrend;
};
```

PublicConstitutionalResult

```
type PublicConstitutionalResult = {
  state: "ORDERED" | "DRIFTING" | "MISALIGNED" | "DISORDERED";
  headline: string;
  summary: string;
  directive: string;
  recommendations: string[];
  escalation?: { required: boolean; label: string };
};
```

FORBIDDEN in Any API Response

The following fields must NEVER appear in any response body:

- `rawScore`
- `severityScore`
- `threshold`
- `weight`
- `matchedSignals`
- `matchedKeywords`
- `arbiterTrace`
- `engineMode`
- `classificationRules`

Chapter 20: IP Protection Architecture

Public Pages

No thresholds, weights, scoring brackets, or axis names appear on any public-facing page. The system uses controlled ambiguity:

- "Proprietary scoring model"
- "Multi-factor evaluation"
- "Dynamic thresholds"

Toolkits

Published toolkits contain conceptual models only. No numeric frameworks, exact scoring ranges, or weighted factor lists appear in downloadable materials.

SessionStorage Encryption

The intelligence spine stored in sessionStorage is encrypted with AES-256-GCM. The encryption key is generated per-session via the Web Crypto API and held only in memory. The key does not persist across page reloads — the spine must be re-fetched from the server.

Chapter 21: Rate Limiting

Architecture

Rate limiting uses a three-tier fallback chain:

1. **Redis (Upstash)** — primary store. Uses atomic increment with TTL via `getRedis()`.
2. **PostgreSQL** — fallback. Uses the `RateLimitBucket` model with `routeKey`, `identityKey`, `count`, and `windowStart` fields. Implemented in `lib/server/security/rate-limit-store.postgres.ts`.
3. **Fail-closed** — if both stores are unavailable, critical routes deny the request.

No In-Memory Production Rate Limiting

There is no in-memory rate limiting in production. The edge middleware uses a per-isolate `Map` for development convenience only.

Chapter 22: Auth Security

Identity Authority

NextAuth/JWT is the sole identity authority. The `getSessionContext()` function in `lib/server/auth/tokenStore.postgres.ts` is the canonical way to resolve a user's identity and access tier.

Cookie Security

The `aol_access` cookie is a session presence indicator only. It does NOT upgrade the user's tier. The server always resolves tier from the database via Prisma.

Integrity Scoring

The diagnostic system includes integrity detection that identifies:

- **Intent flips** — contradictions between stated intent and actual behavior
- **Cost swings** — dramatic changes in stated cost/urgency between assessments
- **Repeated breaches** — recurring pattern violations tracked via `PatternBreakerContract`
- **False authority** — claims of ownership that contradict other inputs

---

PART VII — QUALITY & TESTING

Chapter 23: Testing Strategy

Test Runners

- **Vitest** — unit and integration tests. Configured in `vitest.config.ts`.
- **Playwright** — end-to-end tests. Configured in `playwright.config.ts`. Run separately via `pnpm test:e2e`.

Vitest Configuration

```
export default defineConfig({
  test: {
    environment: 'node',
    globals: true,
    exclude: ['tests/e2e/**', '.next/**', 'node_modules/**'],
    alias: { '@': path.resolve(__dirname, './') },
    maxConcurrency: 5,
    testTimeout: 10000,
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      include: ['lib/ai/**/*.ts'],
      thresholds: {
        lines: 90,
        functions: 95,
        branches: 85,
        statements: 90,
      },
    },
  },
});
```

Test Count

427 vitest-scoped test files (excluding `tests/e2e/**` Playwright specs and `_tests_/components/**`), containing 6,000+ individual test cases. Counts are verified against the tree; the authoritative pass/fail signal is `npx vitest run` in CI. File-count method: all `*.{test,spec}.{ts,tsx}` matched by the default vitest include minus the exclusions in `vitest.config.ts`.

Test Categories

Category	Test?	Notes
Domain logic (`lib/ai/` , `lib/decision/`)	**Always**	Core revenue engine
API routes (`pages/api/`)	**Always**	Input validation, auth, response shape
Utilities (`lib/utils/`)	**Always**	Shared code = shared risk
UI components	**Playwright**	Visual/interaction testing only
Database queries	**Integration**	Test with real DB, seed fixtures
Security functions	**Always**	Rate limiting, auth, encryption

What NOT to Test

- **Generated code** — Contentlayer output, Prisma client, auto-generated types
- **Third-party wrappers** — Thin wrappers around SDKs
- **Static configuration** — `next.config.ts` , `tailwind.config.cjs`

Test Commands Reference

Command	Action
`pnpm test`	Run Vitest (all tests, watch mode)
`pnpm test:ui`	Launch Vitest UI in browser
`pnpm test:coverage`	Run with V8 coverage
`pnpm test:all`	test + check:all + pdf:audit
`pnpm test:integration`	Integration suite (sequential)
`pnpm test:performance`	Performance benchmarks (sequential)
`pnpm validate`	typecheck + lint + test

Chapter 24: Quality Gate

File: `scripts/quality/full-validation.mjs`

The quality gate runs 8 sequential checks. ALL must pass for a CLEAN verdict.

Step	Command	Blocking
1. Prisma validate	`npx prisma validate`	Yes
2. Prisma generate	`npx prisma generate`	Yes
3. TypeScript	`npx tsc --noEmit --pretty false`	Yes
4. PDF audit	`npm run pdf:audit`	Yes
5. MDX integrity	`node scripts/mdx-integrity-check.mjs`	Yes
6. MDX gate	`node scripts/mdx-illegal-jsx-gate.mjs`	Yes
7. Unit tests	`npx vitest run --reporter=verbose`	Yes
8. Build	`npx next build`	Yes

Running

```
node scripts/quality/full-validation.mjs
# or
pnpm quality:full
```

Chapter 25: Definition of Clean

File: `docs/quality/definition-of-clean.md`

Clean means reproducible under full validation, not "it compiled once."

Requirements

1. `git status` understood — no unexplained modifications or untracked files
2. TypeScript passes with NO exclusions (`pnpm exec tsc --noEmit`)
3. Build passes (`pnpm build`)
4. PDF audit passes (`pnpm pdf:audit`)
5. Unit tests pass (`pnpm test:unit`)
6. No untracked TypeScript files without justification
7. No client/server boundary violations
8. **Survived Cut 2** — the change ran through the Five-Cut Loop (Chapter 0): the requirement has a named owner, deletion was genuinely attempted before optimisation, and nothing was sped up or automated that was not first simplified. Net new repository surface area (root files, config variants, `fix-*` scripts) is justified in the PR.

Chapter 25A: Performance Standards

Core Web Vitals Targets

Metric	Target	Measurement
LCP (Largest Contentful Paint)	< 2.5s	75th percentile
FID (First Input Delay)	< 100ms	75th percentile
CLS (Cumulative Layout Shift)	< 0.1	75th percentile
TTFB (Time to First Byte)	< 800ms	Server response
INP (Interaction to Next Paint)	< 200ms	All interactions

Performance Budget

Category	Metric	Budget
Core Web Vitals	LCP	< 2.5s
Core Web Vitals	FID	< 100ms
Core Web Vitals	CLS	< 0.1
API Response	Reads (p95)	< 500ms
API Response	Writes/generation (p95)	< 2000ms
PDF Generation	Per document	< 30s
Build Time	Full production build	< 5 minutes
Bundle Size	First-load JS	< 300KB
Database	Indexed queries (p95)	< 100ms
ISR Revalidation	Content pages	3600s (1 hour)

> **KEY PRINCIPLE**
>
> These budgets are hard limits, not aspirational targets. Any regression past these thresholds blocks deployment until resolved.

ISR Strategy (Incremental Static Regeneration)

Page Type	Revalidation Period	Rationale
Blog/editorial	3600s (1 hour)	Content updates infrequently
Lexicon entries	3600s	Static reference material
Vault pages	3600s	Gated content, rarely changes
Dashboard	0 (dynamic)	Real-time data required
Landing pages	3600s	Marketing content
API routes	N/A (always dynamic)	Server-side only

Manual revalidation available via `revalidatePath()` for critical content updates that cannot wait.

Bundle Optimization

Standalone Output — Next.js standalone mode (`output: "standalone"`) produces a self-contained deployment artifact. No `node_modules` folder in production.

Tree-shaking — All imports must be specific. Never `import * as X from 'library'` .

Dynamic Imports — Heavy components loaded on demand:

```
const HeavyChart = dynamic(() => import('@/components/charts/HeavyChart'), {
  loading: () => <ChartSkeleton />,
  ssr: false,
});
```

Bundle Analysis:

```
pnpm build:analyze   # cross-env ANALYZE=true pnpm build
```

Opens webpack-bundle-analyzer in browser. Use to identify oversized chunks.

Image Optimization

- **Sharp** — Server-side image processing (resize, format conversion)
- **next/image** — Automatic responsive images, WebP/AVIF conversion, lazy loading
- **Assets pipeline** — `pnpm assets:optimize` and `pnpm assets:enterprise` for bulk processing

Redis Caching Layer (Optional)

Redis provides an optional caching layer via Upstash. The system degrades gracefully when Redis is unavailable.

> **KEY PRINCIPLE**

> Redis is a performance enhancement, not a requirement. Every feature must work without Redis. The `REDIS_DISABLED=true` flag causes all cache operations to no-op silently. Never make Redis a hard dependency.

Chapter 25B: Error Handling Philosophy

Standard API Response Shape

```
interface ApiResponse<T = unknown> {
  ok: boolean;
  error?: string;
  data?: T;
  code?: string;
}
```

All API routes return this shape. No exceptions.

Error Categories

Code	Category	HTTP Status	Description
`VALIDATION_ERROR`	ValidationError	400	Input failed Zod schema validation
`AUTH_ERROR`	AuthError	401	Missing or invalid authentication
`FORBIDDEN_ERROR`	ForbiddenError	403	Authenticated but insufficient permissions
`NOT_FOUND`	NotFoundError	404	Resource does not exist
`RATE_LIMIT`	RateLimitError	429	Request limit exceeded
`INTERNAL_ERROR`	InternalError	500	Unhandled server error

Error Handling Principles

1. **Fail loudly in development, fail gracefully in production** — Dev surfaces full stack traces; production returns sanitised messages.
2. **Audit logging on all 500s** — Every internal error is logged with request context, timestamp, and correlation ID.
3. **Never expose stack traces to client** — Production 500 responses contain only generic messages.
4. **Zod validation errors return field-level detail** — Human-readable messages per field for 400 responses.
5. **Rate limit responses include retry-after header** — `Retry-After` header set in seconds on all 429 responses.
6. **Correlation IDs** — Every request gets a unique ID propagated through all log entries.

Chapter 25C: Code Quality

ESLint Configuration

The project uses `eslint.config.mjs` (flat config format). Three configuration layers:

```
export default [
  // 1) Global ignores
  { ignores: [".next/**", "scripts/**", "prisma/**", ...] },

  // 2) Base JS rules
  js.configs.recommended,

  // 3) Next.js rules
  { files: ["pages/**", "components/**", "lib/**", "app/**"],
    plugins: { "@next/next": next },
    rules: { ...next.configs.recommended.rules,
             ...next.configs["core-web-vitals"].rules } },

  // 4) TypeScript rules
  ...tseslint.config({
    files: ["**/*.{ts,tsx}"],
    languageOptions: { parserOptions: { projectService: true } },
    rules: {
      "@typescript-eslint/no-unused-vars": "off",
      "@typescript-eslint/no-explicit-any": "off"
    }
  }),
];
```

Prettier Configuration

```
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": false,
  "printWidth": 80,
  "tabWidth": 2,
  "useTabs": false,
  "endOfLine": "lf"
}
```

> **KEY PRINCIPLE**
>
> Double quotes are the institutional standard. The `endOfLine: "lf"` setting prevents Windows CRLF contamination.

Pre-commit Hooks (Husky)

Husky runs on every `git commit`:

```
# .husky/pre-commit
pnpm mdx:integrity
pnpm mdx:gate
```

> **WARNING**
>
> Never bypass pre-commit hooks with `--no-verify`. If the hooks fail, the MDX content is broken and MUST be fixed before commit.

Code Review Standards

Every PR must satisfy:

1. `pnpm validate` **passes** — Types, lint, and tests green
2. **No new** `any` **types** without justification comment
3. **All API routes have Zod validation** — No unvalidated input
4. **Security-sensitive changes reviewed by Founder** — Auth, encryption, access control
5. **MDX changes pass integrity gate** — Content is code; treat it accordingly
6. **No secrets in diff** — Environment variables only via platform config

---

Chapter 25D: Monitoring Architecture

Monitoring Stack

Layer	Tool	Purpose
Custom Telemetry	/api/telemetry/global	Platform-specific event tracking
Content Telemetry	/api/telemetry/resonance	Content engagement and resonance scoring
Health Checks	Netlify scheduled function	Uptime verification, dependency health
Audit Logging	GovernanceLog, SecurityLog, SystemAuditLog	Full audit trail for compliance
Error Tracking	Sentry (wired)	`sentry.client.config.ts`, `sentry.server.config.ts`, and the `instrumentation.ts` register hook are wired. Safe no-op when no DSN is set (no data sent, logs a disabled notice). Not enabled in production until `SENTRY_DSN` / `NEXT_PUBLIC_SENTRY_DSN` is configured.

Operational Intelligence

Three commercial-health systems sit above raw telemetry and govern whether the platform is allowed to sell.

- **North Star Metrics** (`lib/follow-up/north-star-metrics.ts`) — reduces platform health to three numbers: QER% (Qualified ER Conversion), DAR% (Decision Activation Rate), and TAR% (Truth Acceptance Rate). Each has a hard threshold (QER ≥ 8%, DAR ≥ 60%, TAR ≥ 70%). Breaching a threshold flags the metric `weak` / `critical` and can withdraw `outreachAllowed`.
- **System Integrity Mode** (`lib/follow-up/system-integrity-mode.ts`) — the global kill switch. When QER or TAR fall below critical bounds (with minimum sample sizes), the mode degrades to `degraded` / `critical`, which suppresses the Strategy Room, restricts Executive Reporting access, and surfaces a recalibration notice. It protects reputation by declining to sell rather than shipping misdiagnosis.
- **Founder Dashboard** (`lib/analytics/founder-dashboard.ts`) — the founder operating data layer. It composes North Star Metrics, System Integrity status, and call-funnel analytics into revenue, pipeline, and breakage panels that answer: are we allowed to sell today, where is revenue coming from, and what must be fixed now.

Custom Telemetry Endpoints

Global Telemetry:

```
POST /api/telemetry/global
Body: { event, properties, timestamp }
```

Resonance Telemetry:

```
POST /api/telemetry/resonance
Body: { contentSlug, engagementType, duration, depth }
```

Health Checks

Application Health:

```
GET /api/v2/health
Response: { status: "ok", timestamp, version, services: {...} }
```

System Health Script:

```
pnpm health   # tsx scripts/system/check-system-health.ts
```

Audit Logging

Three audit models capture all security-relevant events:

Model	Purpose	Fields
`GovernanceLog`	Administrative actions	action, performedBy, target, details
`SecurityLog`	Auth/access events	event type, IP, userId, metadata
`SystemAuditLog`	System-wide audit trail	actor, action, resource, severity, metadata

All audit entries are append-only. Deletion requires Founder authorization.

```
await prisma.systemAuditLog.create({
  data: {
    actorType: 'USER',
    actorId: userId,
    action: 'DOWNLOAD_ARTIFACT',
    resourceType: 'pdf',
    resourceId: artifactId,
    severity: 'info',
    metadata: JSON.stringify({ ip, userAgent }),
  }
});
```

Error Tracking Architecture

Error boundaries exist at:

- App-level (`app/error.tsx`)
- Page-level (per-route error boundaries)
- API routes (try/catch with audit logging)

Log Levels

```
LOG_LEVEL=warn   # debug | info | warn | error | critical
```

Level	When Used
`debug`	Verbose development tracing
`info`	Normal operations (startup, requests)
`warn`	Recoverable issues (cache miss, slow query)
`error`	Operation failed, user impacted
`critical`	System integrity compromised

Alert Thresholds

Signal	Severity	Response Time
Build failure	Critical	Immediate — blocks deploy
500 error rate > 1%	Urgent	Investigate within 1 hour
Core Web Vitals regression	Warning	Investigate within 24 hours
Security event (auth anomaly)	Critical	Immediate review
Database connection failure	Critical	Immediate — failover or rollback
PDF generation timeout	Warning	Investigate within 24 hours

PART VIII — OPERATIONS & DEPLOYMENT

Chapter 25E: Dependency Strategy

Package Management

- **pnpm 10.33+** required (enforced via `engines` in package.json)
- **Lock file**: `pnpm-lock.yaml` committed to repository, always used for installs
- **No phantom dependencies**: pnpm strict mode prevents unlisted dep usage

Update Cadence

Category	Cadence	Process
Security patches	Immediately	`pnpm audit fix`, test, deploy
Minor updates	Weekly	Review changelog, update, run full test suite
Major updates	Quarterly	Spike branch, full regression testing, Lead Engineer approval

Forbidden Patterns

- No `*` version ranges in package.json
- No `latest` tags
- No `npm install` (pnpm only)
- No direct `node_modules` manipulation

Heavy Dependencies to Watch

Package	Size Impact	Justification
puppeteer	~200MB+	PDF generation (isolated in serverless function)
@prisma/client	~10MB generated	Type-safe database access (unavoidable)
sharp	~30MB native	Image optimization (production necessity)
contentlayer2	Build-time only	MDX processing (not in production bundle)

Replacement Candidates

If a dependency goes unmaintained, documented alternatives:

- `contentlayer2` → `velite` or custom MDX pipeline
- `next-auth` → `lucia-auth` or `better-auth`
- `resend` → `nodemailer` + SES (self-hosted fallback)

Chapter 26: Build Pipeline

Build Command

Production builds use `pnpm build`, which runs:

1. `pnpm generate:epubs` — generates EPUB artifacts via `tsx scripts/generate-all-epubs.ts`
2. `next build --webpack` — Next.js production build

The Netlify build uses `pnpm build:netlify`, which runs:

1. `pnpm mdx:gate` — validates MDX JSX patterns
2. `pnpm contentlayer:clean` — cleans stale contentlayer artifacts

3. `pnpm mdx:integrity` — validates MDX component integrity
4. `pnpm build:fast` — builds PDF registry, then runs `next build --webpack` with 7GB memory limit

Node Configuration

```
NODE_OPTIONS=--max-old-space-size=7168
```

The 7168 MB (7 GB) heap allocation is required because:

- Contentlayer processes 100+ MDX files with complex frontmatter
- PDF registry generation holds all asset metadata in memory
- Webpack builds the full application graph including all dynamic routes

> **WARNING**
>
> Do not reduce this value below 7168 MB. Build will OOM on the Contentlayer + Next.js combined phase.

Pre-commit Hooks

Two checks run as pre-commit hooks:

- **MDX integrity** (`scripts/mdx-integrity-check.mjs`)
- **MDX gate** (`scripts/mdx-illegal-jsx-gate.mjs`)

Prisma Generate

`prisma generate` runs automatically via `postinstall` in `package.json`. It also runs as step 2 of the quality gate.

Webpack Mode

All builds use `--webpack` flag, not Turbopack.

Standalone Output Mode

`output: "standalone"` in `next.config.ts` produces a self-contained server. The `clean-standalone.mjs` script removes unnecessary files from the standalone output.

Build Failure Troubleshooting

Symptom	Cause	Fix
`MDX gate failed`	Illegal JSX in MDX files	`pnpm fix:mdx` then review
`MDX integrity failed`	Broken component references	Check MDX imports
`JavaScript heap out of memory`	NODE_OPTIONS not set	Ensure `--max-old-space-size=7168`
`Contentlayer error`	Stale cache	`pnpm contentlayer:clean` then rebuild
`Module not found`	Import path error	`pnpm fix:imports`
`Type error`	TypeScript strict violation	`pnpm typecheck` to identify
`Prisma client not generated`	Missing generate step	`pnpm db:generate`

Chapter 27: Deployment

Platform: Netlify

The application deploys to Netlify using `@netlify/plugin-nextjs`. The `netlify.toml` configuration:

```
[build]
  publish = ".next"
  command = "pnpm build:netlify"

[build.environment]
  NODE_VERSION = "20.20.0"
  NODE_OPTIONS = "--max-old-space-size=7168"
  CI = "true"
  NEXT_TELEMETRY_DISABLED = "1"
  NEXT_DISABLE_SOURCEMAPS = "true"
  CONTENTLAYER_DISABLE_WARNINGS = "1"
  NETLIFY_USE_PNPM = "true"
  PNPM_FLAGS = "--frozen-lockfile"
  PNPM_VERSION = "10.29.3"

[[plugins]]
  package = "@netlify/plugin-nextjs"

[functions]
  directory = "netlify/functions_src/functions"
  node_bundler = "nft"
  external_node_modules = ["@prisma/client", ".prisma", "prisma"]
```

Not Vercel

The application does NOT deploy to Vercel. A `vercel.json` file exists in the repository but is not used for production deployment.

Security Headers

Header	Value	Purpose
`X-Content-Type-Options`	`nosniff`	Prevent MIME sniffing
`X-Frame-Options`	`DENY`	Prevent clickjacking
`Referrer-Policy`	`strict-origin-when-cross-origin`	Limit referrer leakage
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	Enforce HTTPS (1 year)
`X-XSS-Protection`	`1; mode=block`	Legacy XSS filter
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	Restrict browser APIs

Redirect Categories

Over 100 redirects configured in `netlify.toml`:

Category 1 — Domain canonicalization:

```
[[redirects]]
  from = "https://abrahamoflondon.org/*"
  to = "https://www.abrahamoflondon.org/:splat"
  status = 301
  force = true
```

Category 2 — Legacy route rewrites:

```
[[redirects]]
  from = "/essays/*"
  to = "/blog/:splat"
  status = 301
```

Category 3 — PDF canonical aliases (100+) ensure every PDF has exactly one canonical URL.

Docker Configuration

For PDF generation and local full-stack development:

```
FROM node:20-slim

RUN apt-get update && apt-get install -y \
    libreoffice chromium fonts-liberation \
    libnss3 libatk-bridge2.0-0 libxcomposite1 \
    --no-install-recommends

ENV PUPPETEER_SKIP_CHROMIUM_DOWNLOAD=true
ENV PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium

WORKDIR /app
COPY package.json pnpm-lock.yaml* ./
RUN npm install --legacy-peer-deps --ignore-scripts
COPY . .

CMD ["tsx", "scripts/pdf/unified-pdf-generator.ts",
     "--scan-content", "--overwrite", "--strict"]
```

Scheduled Functions

```
[[schedule]]
  path = "/api/cleanup-download-tokens"
  schedule = "0 2 * * *"   # Daily at 2 AM UTC
```

Scheduled functions handle:

- Download token cleanup (expired tokens purged daily)
- Stale session removal
- Audit log rotation

Deploy Hooks

Netlify deploy hooks trigger builds from external events:

- Content CMS publish → webhook → rebuild
- Manual trigger via Netlify dashboard
- `pnpm deploy` — validates links, commits, pushes (auto-deploys on push to main)

Environment Setup

Context	`NEXT_PUBLIC_APP_ENV`	`NEXT_PUBLIC_SITE_URL`
Production	`production`	`https://www.abrahamoflondon.org`
Deploy Preview	`staging`	Auto-generated Netlify URL
Local	`development`	`http://localhost:3000`

Required Environment Variables

Variable	Service	Required
`DATABASE_URL`	Neon PostgreSQL	Yes
`NEXTAUTH_SECRET`	NextAuth JWT signing	Yes
`NEXTAUTH_URL`	NextAuth callback URL	Yes
`STRIPE_SECRET_KEY`	Stripe payments	Yes
`STRIPE_WEBHOOK_SECRET`	Stripe webhooks	Yes
`RESEND_API_KEY`	Email delivery	Yes
`UPSTASH_REDIS_REST_URL`	Redis cache	Optional
`UPSTASH_REDIS_REST_TOKEN`	Redis auth	Optional

Rollback Procedure

1. Open Netlify dashboard, find last known-good deploy
2. Click "Publish deploy" on that entry
3. Verify site is restored
4. Investigate and fix the broken commit on a branch

> **KEY PRINCIPLE**
>
> Netlify retains all previous deploys indefinitely. Rollback is instant — no rebuild required. Always roll back first, investigate second.

Chapter 28: Environment Management

Database: PostgreSQL Only

The production database is PostgreSQL via Neon. There is no SQLite in the production data path.

Redis: Optional

Redis via Upstash is optional. The system degrades gracefully:

1. Redis available -> used for rate limiting and caching
2. Redis unavailable -> PostgreSQL fallback for rate limiting
3. Both unavailable -> fail-closed on critical routes, pass-through on non-critical

Variable Categories (18 Categories)

#	Category	Count	Critical?
1	Application	7	Yes
2	Database	2	Yes
3	Redis	5	No (optional)
4	Authentication	8	Yes
5	Admin	8	Yes
6	Cron/Internal	3	Yes
7	Brand/Identity	8	Yes
8	Email	7	Partial
9	Inner Circle	4	Yes
10	Enterprise/Diagnostics	5	Yes
11	Sovereign/OGR	3	Yes
12	Payments (Stripe)	2	No (optional)
13	AI	1	No (optional)
14	OAuth Integrations	5	No (optional)
15	PDF	4	Yes
16	Security	2	Yes
17	Feature Flags	6	No (defaults)
18	Development	3	No (dev only)

Secrets Rotation

1. Generate new secret value
2. Update in Netlify environment variables
3. Trigger redeploy
4. Revoke old secret value
5. Log rotation in governance audit

Feature Flags

Flag	Default	Effect When `true`
`ENABLE_ANALYTICS`	`false`	Activates analytics
`ENABLE_PDF_GENERATION`	`true`	Allows runtime PDF creation
`ENABLE_EMAIL_NOTIFICATIONS`	`false`	Enables Resend email delivery
`SKIP_DB`	`false`	Bypasses all database operations
`REDIS_DISABLED`	`true`	Disables Redis cache layer
`DEBUG_CONTENTLAYER`	`false`	Verbose Contentlayer logging

Graceful Degradation

Service Missing	Behavior
Redis	Cache operations no-op, all data fetched fresh
Stripe	Payment UI hidden, checkout disabled
OpenAI	AI features return fallback/static responses
Email (Resend)	Notifications queued but not sent

Chapter 29: Monitoring

SecurityLog

The `SecurityLog` Prisma model records security events: LOGIN_SUCCESS, LOGIN_FAILURE, MFA_CHALLENGE, PASSWORD_CHANGE, UNAUTHORIZED_ACCESS. Each event captures the actor, HTTP method, path, IP address, and user agent.

DiagnosticJourney — Evidence Spine

The `DiagnosticJourney` model is the central evidence spine. Every stage from Fast Diagnostic through Post-Strategy Room persists its evidence to the journey store via `persistDiagnosticStage()` in `lib/diagnostics/journey-store.ts`.

Evidence flow:

```
Fast Diagnostic → persistSpineToJourney() → DiagnosticJourney
Purpose Alignment → persistDiagnosticStage("purpose_alignment") → DiagnosticJourney
Constitutional → persistDiagnosticStage("constitutional") → DiagnosticJourney
Team Assessment → persistDiagnosticStage("team") → DiagnosticJourney
Enterprise Assessment → persistDiagnosticStage("enterprise") → DiagnosticJourney
Executive Reporting → persistDiagnosticStage("executive_reporting") → DiagnosticJourney
Strategy Room → persistDiagnosticStage("strategy_room") → DiagnosticJourney
```

Each stage stores a `payload` (the stage-specific result data), `snapshot` (core metrics + tensions + escalation level), `evidenceNodes`, `decisionObject`, and `tensions`.

Evidence retrieval:

Loader	File	Purpose
`getDiagnosticJourney()`	`lib/diagnostics/journey-store.ts`	Load full journey with all stages
`loadPurposeAlignmentEvidence()`	`lib/alignment/evidence-loader.ts`	Load PA-specific evidence fields
`loadLatestFinancialExposure()`	`lib/product/financial-exposure-persistence.ts`	Load latest FE snapshot
`loadLatestCostOfInactionProjection()`	`lib/product/financial-exposure-persistence.ts`	Load latest cost-of-inaction projection
`resolveLadderContext()`	`lib/diagnostics/ladder-context-resolver.ts`	Load ladder context from Constitutional/Team/Enterprise

Evidence rendering — Governed Memory System:

All evidence is rendered via `GovernanceEvidenceCarryForward` (`components/strategy-room/GovernanceEvidenceCarryForward.tsx`) which displays `GovernedMemoryItem[]` with:

- Source label: `"{confidenceLabel} in {sourceSurfaceLabel}"` (e.g. `"CAPTURED in Purpose Alignment"`)
- Date: formatted via `formatCapturedDate()`
- Status: `ACTIVE`, `UNRESOLVED`, `STALE`, `SUPERSEDED`, `RESOLVED`, `SUPPRESSED`
- Safety: `isMemoryDisplaySafe()` — suppressed items show explicit reason

Evidence converters (raw → GovernedMemoryItem[]):

Converter	File	Source Surface
`convertPurposeAlignmentToGovernedMemory()`	`lib/alignment/evidence-loader.ts`	PURPOSE_ALIGNMENT
`convertFinancialExposureToGovernedMemory()`	`lib/product/financial-exposure-persistence.ts`	FAST_DIAGNOSTIC or EXECUTIVE_REPORTING
`buildGovernedMemoryFromEvidenceCapture()`	`lib/product/governed-memory-presenter.ts`	Configurable
`buildGovernedMemoryFromEvidenceStages()`	`lib/product/governed-memory-presenter.ts`	Per-stage

Evidence surfaces (where evidence is rendered):

Surface	PA Evidence	FE Evidence	Source Labels	Dates
Executive Reporting UI	✅	✅	✅	✅
Strategy Room Entry	✅	✅	✅	✅
Strategy Room Session	✅	✅	✅	✅
Return Brief UI	✅	✅	✅	✅
Decision Centre	✅	✅	✅	✅
Oversight Brief UI	✅	✅	✅	✅

Closure status: 80/80 material fields across 8 ladder stages are CLOSED_RENDERED, CLOSED_SIGNALLED, or CLOSED_SUPPRESSED. See `docs/product/whole-ladder-evidence-spine-closure-register.md` for the definitive field-by-field register.

Financial Exposure Persistence Policy:

Defined in `lib/product/financial-exposure-persistence.ts`. Persists computed numeric exposure snapshot when generated:

- `userCostOfDelayText` — user's free-text description
- `estimatedFinancialExposure` — computed numeric estimate
- `exposureBand` — low/moderate/high/critical/undisclosed
- `exposureBasis` — input parameters used for calculation
- `computedAt`, `sourceSurface`, `schemaVersion`

Also persists costOfInaction projections (qualitative horizon narratives) via `persistCostOfInactionProjection()`.

Every rendered FE item includes the caveat: *"This is an estimate based on diagnostic inputs and has not been independently verified."*

Execution Tracking

- `DecisionJourneyEvent` — records decision lifecycle events with timestamps
- `AuditEvent` — general audit trail for system actions
- `SystemAuditLog` — system-level audit events

- `OperationalIncident` — tracks operational incidents with severity and resolution status
- `ServiceLevelSnapshot` — captures SLA metrics at regular intervals

PART IX — SECURITY

Chapter 30: Security Architecture

Defense in Depth

```
┌─────────────────────────────────────────────────────────┐
│  NETWORK LAYER                                          │
│  - HTTPS enforced (HSTS 1yr)                            │
│  - CORS whitelist                                       │
│  - Rate limiting (Redis primary, Postgres fallback)     │
├─────────────────────────────────────────────────────────┤
│  APPLICATION LAYER                                      │
│  - CSRF protection (synchronizer token)                 │
│  - Input validation (Zod schemas on ALL routes)         │
│  - Output encoding (React default + DOMPurify)          │
│  - Content Security Policy headers                      │
├─────────────────────────────────────────────────────────┤
│  DATA LAYER                                             │
│  - Access tier enforcement (8 levels)                   │
│  - Entitlement system (TIER, PRODUCT, ARTIFACT)         │
│  - Encryption at rest (Neon TLS, AES-256-GCM for tokens)│
├─────────────────────────────────────────────────────────┤
│  AUDIT LAYER                                            │
│  - All access attempts logged                           │
│  - GovernanceLog, SecurityLog, SystemAuditLog           │
│  - Append-only (no deletion without Founder auth)       │
└─────────────────────────────────────────────────────────┘
```

Rate Limiting by Endpoint

Rate limiting is enforced in `proxy.ts` per-IP per-pathname, using tiered configuration:

Endpoint Type	Limit	Window	Source
API General	200 requests	60 seconds	`proxy.ts`
Admin (`/admin/`)	60 requests	60 seconds	`proxy.ts`
Constitutional (`/constitutional/`)	30 requests	60 seconds	`proxy.ts`
Sovereign (`/sovereign/`)	20 requests	60 seconds	`proxy.ts`
Auth (`/auth/`)	10 requests	60 seconds	`proxy.ts`
Contact form	5 requests	1 hour	Route-level
Downloads	20 requests	1 hour	Route-level

> **WARNING**
>
> Proxy rate limiting uses in-memory storage (per-isolate). It does not persist across serverless cold starts or share across Netlify edge instances. For strict global enforcement, route-level rate limiting via Upstash Redis is used on critical paths (diagnostics capture, inner-circle verify, email endpoints).

CSRF Protection

Synchronizer token pattern:

1. Server generates CSRF token on page load
2. Token embedded in form/stored in meta tag
3. All state-changing requests must include token
4. Server validates token matches session

Input Validation

> **KEY PRINCIPLE**
>
> Every API route validates input with Zod. No raw `request.body` access without schema validation. This is non-negotiable.

Chapter 31: Authentication Security

JWT Configuration

Parameter	Value
Algorithm	HS256
Expiry	30 days
Secret	`NEXTAUTH_SECRET` (min 32 chars)

Cookie Security

Attribute	Value	Purpose
`HttpOnly`	`true`	Prevents JavaScript access
`Secure`	`true` (production)	HTTPS only
`SameSite`	`Strict`	Prevents CSRF via cookies

OAuth Token Encryption

OAuth tokens (Google Calendar, Slack) are encrypted at rest using AES-256-GCM:

1. OAuth callback receives access/refresh tokens
2. Tokens encrypted with AES-256-GCM before database storage
3. Decrypted only at point of use (API call to provider)
4. IV is unique per encryption operation

Password Storage

Method	Use Case
Argon2	All user passwords
bcrypt	Legacy (migration path)
Plaintext	NEVER

MFA Implementation

TOTP-based MFA via `@otplib/preset-default`:

```
model MfaSetup {
  id        String    @id @default(cuid())
  memberId  String
  method    MfaMethod @default(totp)
  secret    String    // Encrypted TOTP secret
  verified  Boolean   @default(false)
  createdAt DateTime  @default(now())
}
```

Session Management

- Server-side session validation on every request
- Token rotation on privilege escalation
- Automatic expiry after `AOL_SESSION_TTL_DAYS` (default: 30)
- Revocation logged to `SecurityLog`

Admin Bootstrap

Initial admin creation uses environment whitelist:

```
ADMIN_ALLOWED_EMAILS=admin@abrahamoflondon.org
ADMIN_USER_EMAIL=admin@abrahamoflondon.org
```

Only emails in `ADMIN_ALLOWED_EMAILS` can be granted `ADMIN` or `OWNER` roles.

Chapter 32: Data Protection

Access Tier Enforcement

Eight access levels, strictly hierarchical:

Level	Enum Value	Who
0	`public`	Anyone
1	`member`	Registered users
2	`inner_circle`	Inner Circle members
3	`restricted`	Specific entitlement holders
4	`client`	Active clients
5	`legacy`	Legacy program members
6	`architect`	System architects
7	`owner`	Founder only
8	`top_secret`	Founder + explicit grant

Entitlement System

```
model Entitlement {
  id        String            @id @default(cuid())
  userId    String
  type      EntitlementType   // TIER | PRODUCT | ARTIFACT
  key       String
  status    EntitlementStatus // ACTIVE | REVOKED | EXPIRED
  issuedAt  DateTime
  expiresAt DateTime?
  issuedBy  String?
  revokedBy String?
  reason    String?
}
```

Audit Trail

```
model AccessAuditLog {
  id           String     @id @default(cuid())
  userId       String?
  resourceType String
  resourceId   String
  accessType   AccessType // VIEW | DOWNLOAD | METADATA
  granted      Boolean
  reason       String?
  ip           String?
  userAgent    String?
  createdAt    DateTime   @default(now())
}
```

> **KEY PRINCIPLE**
>
> Audit logs are evidence. They are append-only, indexed by user and resource, and retained indefinitely. No deletion without written Founder authorization.

Encryption at Rest

Data	Encryption
Database (Neon)	TLS in transit, encrypted storage
OAuth tokens	AES-256-GCM (application-level)
PDF watermarks	HMAC-SHA256 (integrity)
Session tokens	Random + hashed storage
MFA secrets	Encrypted column

Anonymization

Three salt values support data anonymization:

```
AOL_HASH_SALT=<random-secret>       # General hashing
ANONYMITY_SALT=<random-secret>      # User identity anonymization
DENYLIST_PEPPER=<random-secret>     # Denylist matching without storing emails
```

Data Retention Policies

Data Type	Retention	Justification
Audit logs	Indefinite	Legal/compliance requirement
Session data	90 days after expiry	Cleanup via scheduled function
Download tokens	72 hours	Purged daily at 2 AM UTC
Telemetry events	365 days	Annual performance review
User data	Until deletion request	GDPR compliance
Assessment data	5 years	Client contract requirement

Chapter 32A: ZTHVF Security Validation Framework

Overview

The Zero-Trust Hostile Validation Framework (ZTHVF) is the system's formal security verification protocol. It converts security from belief to evidence by requiring every security claim to be runtime-provable, adversarially tested, and forensically observable.

ZTHVF is not a penetration test. It is a convergence operation that eliminates unverifiable assumptions and produces forensic-grade proof of system behaviour under attack.

Verification Levels

Level	Name	Meaning
SV	STATICALLY VERIFIED	Source code inspected
BV	BUILD VERIFIED	Build succeeded without errors
RV	RUNTIME VERIFIED	Executed and observed locally
DV	DEPLOYMENT VERIFIED	Executed on deployed infrastructure
AV	ADVERSARIALLY VERIFIED	Attack attempted against running system
FV	FORENSICALLY VERIFIED	Audit logs confirm expected events

Static Gates (Pre-Runtime)

Three static gates must pass before any runtime validation:

Gate	Script	Requirement
Dependency audit	`pnpm audit --prod`	0 critical/high. Moderate must be patched or documented unreachable
Client bundle audit	`scripts/security/audit-client-bundle-secrets.mjs`	Zero banned patterns in `.next/static/`
Public IP exposure audit	`scripts/security/audit-public-ip-exposure.mjs`	Zero IP-exposing terms in public-facing source

Hostile Test Suite

The adversarial test suite lives at `scripts/security/red-team-smoke.mjs` and covers:

Category	Tests
Auth bypass	Cron without secret, admin without session, forged tokens
Input abuse	Malformed JSON, oversized payload, schema bypass, proto pollution
Access control	Cross-user access, IDOR, null ownership, download without entitlement
Rate limiting	Burst attacks, replay floods
Payment	Price injection, entitlement injection, webhook replay
Bundle exposure	Secret/key scan of client-shipped JavaScript

Artifacts

ZTHVF produces the following documentation in `docs/security/zthvf/`:

File	Purpose
`surface-map.json`	Machine-readable attack surface summary
`route-inventory.md`	Every API route with auth classification
`public-asset-map.md`	All publicly accessible static files
`trust-boundaries.md`	Cookie, token, session, entitlement mechanisms
`middleware-coverage.md`	Proxy execution order and gap analysis

Release Gate

No system may be declared market-ready unless:

- All static gates pass
- Build compiles with zero TypeScript errors
- Hostile test suite produces zero exploitable findings
- All deployment-dependent tests pass on staging/production
- Forensic logs confirm denial events for every hostile attempt

> **KEY PRINCIPLE**
>
> The words "secure", "hardened", "protected", and "market-ready" are banned in engineering communication unless accompanied by a specific verification level (SV/BV/RV/DV/AV/FV) and evidence reference. Security is proved, not declared.

PART X — CANONICAL CONTRACT REFERENCE

Chapter 33: Intelligence Spine Contract

File: `lib/decision/intelligence-spine.ts`

The IntelligenceSpine is the single contract object that flows through every diagnostic stage.

```
type IntelligenceSpine = {
  id: string;
  userId?: string;
  email?: string;

  case: CaseObject;
  c3: SpineC3;
  deterministic: DeterministicOutput;
  synthesis: GovernedSynthesis | null;
  forecast: DefaultPathForecast;
  memory: CaseMemoryResult | null;
  kernel: DecisionKernelOutput | null;
  stakeholders: StakeholderMap;

  currentStage: SpineStage;
  history: SpineEvent[];
  createdAt: string;
  updatedAt: string;
};
```

SpineC3

```
type SpineC3 = C3Score & {
  tier: "HARD_RECOVERY" | "SOFT_RECOVERY" | "FULL_SYNTHESIS";
  confidenceBand: "low" | "medium" | "high";
};
```

DeterministicOutput

```
type DeterministicOutput = {
  conditionClass: "authority" | "definition" | "execution" | "instability";
  signal: SignalDefinition;
  contradictionSet: string[];
  blockerClass: string;
};
```

Stage Progression

```
fast_diagnostic -> constitutional -> team -> enterprise ->
executive_reporting -> strategy_room -> outcome_verification
```

Each stage appends a `SpineEvent` to the `history` array. Stages are append-only — regression is detected by `validateIntelligenceSpine()`.

Validation

`validateIntelligenceSpine()` checks for:

- Case ID and decision text presence
- C3 score bounds (0-1)
- Tier presence
- HARD_RECOVERY must not have synthesis
- History must be append-only (no stage regression)
- Deterministic condition class must exist
- Timestamps must exist

`assertValidSpine()` throws on BLOCK-level errors. Used at trust boundaries (API routes, DB persistence).

Chapter 34: Constitutional Derivation

The constitutional diagnostic system has a single canonical source in `lib/diagnostics/`. Key files:

- `lib/diagnostics/constitutional-diagnostic-derivation.ts` — canonical derivation logic
- `lib/diagnostics/constitutional-bridge.ts` — bridge between constitutional and fast diagnostic systems
- `lib/diagnostics/constitutional-handoff.ts` — handoff protocol between stages
- `lib/diagnostics/public-constitutional-result.ts` — public DTO definition and sanitizer

The `toPublicResult()` function strips all scoring, thresholds, signals, and engine internals before the API boundary.

Chapter 35: Scoring API Contract

POST /api/diagnostics/score

Request:

```
{
  answers: Record<string, string>,
  committed: boolean
}
```

Validated with zod. The `answers` record must contain at minimum a `decision` field with 10+ characters.

Response (success): Returns `FastDiagnosticResult` (see Chapter 19).

Pipeline:

1. Parse and validate request body
2. Create CaseObject from answers
3. Check for contradiction (pre-synthesis)
4. Score C3 fidelity
5. Build deterministic output
6. Forecast default path
7. Synthesize (LLM + arbiter tournament)
8. Compute Cost of Inaction
9. Persist Financial Exposure snapshot (`persistFinancialExposureSnapshot`)
10. Persist Cost of Inaction projection (`persistCostOfInactionProjection`)
11. Assess Execution Failure
12. Compute Authority Index
13. Create Decision Memory record
14. Summarize Decision Memory trend
15. Create Intelligence Spine
16. Persist spine to DiagnosticJourney
17. Return sanitized FastDiagnosticResult

PART XI — PDF PIPELINE

Chapter 36: PDF Architecture

Five Generation Paths

Path	Technology	Use Case	Pros	Cons
React PDF	`@react-pdf/renderer`	Diagnostic reports, data-heavy documents	Full React component model, dynamic layouts	No CSS support, limited styling
jsPDF	`jspdf + jspdf-autotable`	Simple certificates, receipts	Fast, no server deps, client-side capable	Limited layout control
Puppeteer	`puppeteer`	Complex visual layouts, branded content	Full CSS/HTML support	Heavy, requires Chrome, slow
LibreOffice	`libreoffice --headless`	DOCX to PDF conversion	Handles complex Word documents	Requires system binary, slow
md-to-pdf	`md-to-pdf`	Markdown content pages, documentation	Simple input format, consistent output	Limited styling

When to Use Which Path

```
Decision tree:

Is the source Markdown?
  → Yes: md-to-pdf
  → No: Continue

Is the source a .docx file?
  → Yes: LibreOffice
  → No: Continue

Does it need complex CSS layouts (grids, gradients, precise positioning)?
  → Yes: Puppeteer
  → No: Continue

Is it data-driven with tables, charts, dynamic sections?
  → Yes: React PDF
  → No: Continue

Is it a simple single-page document (certificate, receipt)?
  → Yes: jsPDF
  → No: React PDF (default)
```

> **KEY PRINCIPLE** React PDF is the default. Only deviate when the content requirements demand a different renderer.

Unified PDF Generator

```
interface GenerationRequest {
  source: string;
  renderer: RendererType;   // REACT_PDF | JSPDF | PUPPETEER | LIBREOFFICE | MD_TO_PDF
  outputPath: string;
  options: RendererOptions;
  metadata: PDFMetadata;
}

export async function generatePDF(request: GenerationRequest): Promise<GenerationResult> {
  // 1. Validate request
  // 2. Select renderer
  // 3. Pre-process source
  // 4. Generate
  // 5. Post-process (watermark, metadata, fingerprint)
  // 6. Write output
  // 7. Register in manifest
}
```

Registry System

The PDF registry tracks all generated PDFs:

```
interface PDFRegistryEntry {
  id: string;
  filename: string;
  path: string;
  renderer: RendererType;
  category: string;       // diagnostic, alignment, certificate, content
  generatedAt: string;
  size: number;
  hash: string;           // SHA-256 of content
  metadata: PDFMetadata;
  governance: GovernanceStatus;
}
```

> **WARNING** The registry file `pdf-registry.generated.ts` is AUTO-GENERATED. Never edit it manually. Run `pnpm pdf:registry` to regenerate.

Governance Rules

Rule	Requirement	Validation
Naming	`[category]-[descriptor]-[version].[ext]`	Regex pattern match
Size	Maximum 10MB per file	File size check
Header	Must contain institutional header with logo	First-page analysis
Metadata	Title, author, creation date required	PDF metadata extraction
Fonts	Only approved font families	Font embedding check
Hash	SHA-256 recorded in registry	Integrity verification

Font Management

```
export const FONT_REGISTRY = {
  Inter: {
    family: 'Inter',
    weights: { regular, medium, semibold, bold },
    fallback: 'Helvetica',
  },
  'Cormorant Garamond': {
    family: 'Cormorant Garamond',
    weights: { regular, medium, semibold, bold },
    fallback: 'Times-Roman',
  },
  'JetBrains Mono': {
    family: 'JetBrains Mono',
    weights: { regular, medium, bold },
    fallback: 'Courier',
  },
};
```

Watermarking and Fingerprinting

```
interface WatermarkOptions {
  text?: string;           // Visible watermark text
  fingerprint: string;     // Unique identifier embedded invisibly
  opacity?: number;        // 0-1 for visible watermark
  position?: 'diagonal' | 'footer' | 'header';
}
```

> **KEY PRINCIPLE** Every PDF generated by the platform carries an invisible fingerprint. This enables tracing leaked documents back to the specific user and generation event. The fingerprint is

> a SHA-256 hash of `userId + generationTimestamp + documentId`.

Enterprise PDF Generation

Enterprise PDFs are generated via API routes and Netlify functions:

```
// app/api/pdf/enterprise/route.ts
export async function POST(req: NextRequest) {
  const session = await getAuthSession();
  if (!session) return unauthorized();

  const body = await req.json();
  const { campaignId, format } = enterprisePdfSchema.parse(body);

  if (!await hasEntitlement(session.user.id, 'TIER', 'ENTERPRISE')) {
    return forbidden();
  }

  const jobId = await queuePdfGeneration({
    type: 'enterprise_assessment',
    campaignId,
    format,
    userId: session.user.id,
    renderer: 'REACT_PDF',
    watermark: {
      fingerprint: generateFingerprint(session.user.id, campaignId),
    },
  });

  return json({ ok: true, data: { jobId }, code: 202 });
}
```

Chapter 37: PDF Operations

Commands Reference

Command	Purpose
`pnpm pdf:generate`	Generate PDFs from source files
`pnpm pdf:registry`	Rebuild the PDF registry
`pnpm pdf:audit`	Run full audit (duplicates, links, governance)
`pnpm pdf:audit:duplicates`	Find duplicate PDFs by hash
`pnpm pdf:audit:canonicals`	Verify canonical references
`pnpm pdf:audit:links`	Check internal/external links
`pnpm pdf:audit:governance`	Validate governance compliance
`pnpm pdf:repair`	Detect and fix corrupt/empty PDFs
`pnpm pdf:manifest`	Regenerate manifest.json
`pnpm pdf:validate`	Validate specific PDF(s)
`pnpm pdf:clean`	Remove orphaned/invalid PDFs
`pnpm pdf:stats`	Print registry statistics

Generation Workflow

```
scan → generate → validate → register
```

1. **Scan** — Identify source files requiring PDF generation
2. **Generate** — Route through appropriate renderer
3. **Validate** — Check governance compliance
4. **Register** — Add to registry and manifest

Audit Workflow

```
pnpm pdf:audit              # Complete audit
pnpm pdf:audit:duplicates   # SHA-256 hash comparison
pnpm pdf:audit:canonicals   # Canonical URL verification
pnpm pdf:audit:links        # Dead link detection
pnpm pdf:audit:governance   # Rule compliance
```

Repair Workflow

```
pnpm pdf:repair:detect   # Identify corrupt files
pnpm pdf:repair:fix      # Attempt automatic repair
# Strategies: empty files → regenerate, truncated → regenerate,
# invalid header → repair metadata, unfixable → flag for manual
```

PART XII — EXTERNAL INTEGRATIONS

Chapter 38: Integration Catalog

Resend (Email)

```
import { Resend } from 'resend';

export const resend = new Resend(process.env.RESEND_API_KEY);

enum EmailType {
  CONTACT = 'CONTACT',
  INNER_CIRCLE = 'INNER_CIRCLE',
  INVITE = 'INVITE',
  ENTERPRISE = 'ENTERPRISE',
  SYSTEM = 'SYSTEM',
  TRANSACTIONAL = 'TRANSACTIONAL',
}

// Template system: lib/email/templates/
// ├── contact.tsx, inner-circle.tsx, invite.tsx, enterprise.tsx

export async function sendEmail(options: SendEmailOptions): Promise<void> {
  const template = getTemplate(options.type);
  const html = render(template(options.data));

  await resend.emails.send({
    from: options.from ?? 'Abraham of London <info@abrahamoflondon.org>',
    to: options.to,
    subject: options.subject,
    html,
    tags: [{ name: 'type', value: options.type }],
  });
}
```

Stripe (Payments)

Key integration points:

Component	Purpose
`lib/stripe/client.ts`	Stripe SDK instance
`lib/stripe/webhooks.ts`	Event handlers by type
`lib/stripe/customers.ts`	Customer create/retrieve
`lib/stripe/subscriptions.ts`	Subscription lifecycle
`pages/api/webhooks/`	Webhook receiver

Algolia (Search)

```
import algoliasearch from 'algoliasearch';

const client = algoliasearch(
  process.env.ALGOLIA_APP_ID!,
  process.env.ALGOLIA_ADMIN_KEY!
);

export const indices = {
  content: client.initIndex('content'),
  lexicon: client.initIndex('lexicon'),
  diagnostics: client.initIndex('diagnostics'),
};
```

Anthropic / OpenAI (AI)

```
import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY!,
});

export async function generateNarrative(
  context: NarrativeContext
): Promise<string> {
  const message = await anthropic.messages.create({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 2048,
    system: SYSTEM_PROMPT,
    messages: [
      { role: 'user', content: buildNarrativePrompt(context) },
    ],
  });
  return extractTextContent(message);
}
```

> **WARNING** AI-generated content must always be clearly marked as such in user-facing surfaces.

Neon (Database)

Connection pooling is managed by Neon's serverless driver. No PgBouncer required.

Upstash (Redis / Rate Limiting)

```
import { Redis } from '@upstash/redis';

export const redis = Redis.fromEnv();
// Expects UPSTASH_REDIS_REST_URL and UPSTASH_REDIS_REST_TOKEN
```

PART XIII — SERVERLESS FUNCTIONS

Chapter 39: Netlify Functions

Function Inventory

Function	Trigger	Purpose
`contact`	HTTP	Process contact form submissions
`download`	HTTP	Secure file download with signed URLs
`subscribe`	HTTP	Newsletter / mailing list subscription
`health-check`	HTTP/Cron	Platform health verification
`cleanup`	Cron	Expired session and orphan cleanup
`test-email`	HTTP	Email delivery verification (staging only)
`ping`	HTTP	Simple availability check

Function Architecture

```
netlify/functions_src/functions/
├── contact.ts
├── download.ts
├── subscribe.ts
├── health-check.ts
├── cleanup.ts
├── test-email.ts
├── ping.ts
└── _shared/
    ├── _utils.ts       ← Common utilities
    └── _email.ts       ← Email sending (Resend)
```

Scheduled Functions (Cron Patterns)

```
# netlify.toml
[functions."health-check"]
schedule = "*/5 * * * *"    # Every 5 minutes

[functions."cleanup"]
schedule = "0 3 * * *"      # Daily at 03:00 UTC
```

```
import { schedule } from '@netlify/functions';

export const handler = schedule('0 3 * * *', async (event) => {
  const expiredSessions = await cleanExpiredSessions();
  const orphanedArtifacts = await cleanOrphanedArtifacts();
  return { statusCode: 200 };
});
```

Netlify Configuration Detail

```
# netlify.toml — Functions section

[functions]
  directory = "netlify/functions_src/functions"
  node_bundler = "nft"
  external_node_modules = ["@prisma/client", ".prisma", "prisma"]

[functions."*"]
  timeout = 30
```

Bundle Optimization

Prisma is externalised to avoid bundling the query engine into each function:

```
external_node_modules = ["@prisma/client", "prisma"]
```

This ensures:

- Function bundles remain small (< 50MB limit)
- Prisma's native query engine is loaded at runtime
- Cold start times are minimised

> **KEY PRINCIPLE** Netlify functions have a 50MB bundle limit and 30s configured timeout. Keep functions lean — heavy computation belongs in background functions.

Environment Variable Pass-Through

All environment variables set in the Netlify dashboard are available to functions. Key variables required:

```
DATABASE_URL          — Neon connection string
RESEND_API_KEY        — Email sending
STRIPE_SECRET_KEY     — Payment verification
STRIPE_WEBHOOK_SECRET — Webhook signature validation
UPSTASH_REDIS_REST_URL    — Rate limiting
UPSTASH_REDIS_REST_TOKEN  — Rate limiting auth
ALLOWED_ORIGIN        — CORS
```

PART XIV — DEVELOPER OPERATIONS

Chapter 40: Local Development Setup

Prerequisites

- Node.js >= 20.0.0 (exact: 20.20.0 for parity with CI)
- pnpm >= 10.33 (install: `npm install -g pnpm@latest`)
- Git 2.40+
- PostgreSQL connection string (Neon or local)
- (Optional) LibreOffice — for PDF generation
- (Optional) Docker — for containerized PDF pipeline

Setup Steps

```
# 1. Clone the repository
git clone <repo-url>
cd aol-check-visual

# 2. Install dependencies
pnpm install

# 3. Configure environment
cp .env.example .env
# Edit .env with your DATABASE_URL, NEXTAUTH_SECRET, etc.

# 4. Generate Prisma client
npx prisma generate

# 5. Push schema to database (or run migrations)
npx prisma db push
# or: npx prisma migrate deploy

# 6. Seed the database (optional)
pnpm db:seed

# 7. Start development server
pnpm dev
```

The dev server runs on `http://localhost:3000` by default.

Environment File

The `.env` file must contain at minimum:

```
DATABASE_URL=postgresql://user:password@host:5432/database?sslmode=require
NEXTAUTH_SECRET=<random-secret>
NEXTAUTH_URL=http://localhost:3000
```

Windows-Specific Notes

```
IS_WINDOWS=true   # Set in .env on Windows
```

Issue	Solution
Path separators	Scripts normalize `\` to `/` when `IS_WINDOWS=true`
LibreOffice path	`C:\Program Files\LibreOffice\program\soffice.com`
Line endings	Git: `core.autocrlf=input`. Prettier enforces LF.
Long paths	Enable: `git config --system core.longpaths true`
Encoding	`pnpm fix:encoding` resolves UTF-8 BOM issues

```
pnpm dev:windows   # cross-env IS_WINDOWS=true contentlayer2 build && next dev
pnpm fix:windows   # Auto-fix Windows path issues
```

Common Setup Issues

Problem	Fix
`prisma generate` fails	`pnpm fix:prisma` (re-runs generate)
Contentlayer cache corrupt	`pnpm contentlayer:clean && pnpm contentlayer:build`
Port 3000 in use	Kill process or use `pnpm preview` (port 3001)
Node version mismatch	Use nvm: `nvm use 20`
pnpm lockfile conflict	`pnpm install --frozen-lockfile` or `pnpm reinstall`

Chapter 41: Development Workflow

Branch Strategy

Branch from `main`. The main branch is the production branch. No develop branch. No staging branch. Ship directly.

Commit Conventions

All commits use a prefix convention:

Prefix	Use
`Feature:`	New functionality
`Fix:`	Bug fix
`Refactor:`	Code restructure (no behavior change)
`Content:`	MDX/content changes
`Style:`	CSS/visual changes
`Docs:`	Documentation updates
`Test:`	Test additions/changes
`Build:`	Build system changes
`Deploy:`	Deployment configuration
`Security:`	Security fixes/hardening

Before Pull Request

Run the full quality gate:

```
node scripts/quality/full-validation.mjs
```

All 8 checks must pass. Do not submit a PR with failing checks.

Code Review Checklist

- ☐ Types correct (no new `any` without comment)
- ☐ Zod validation on all API inputs
- ☐ Error handling follows `{ok, error, data, code}` pattern
- ☐ No secrets in code
- ☐ Access tier enforcement for new routes
- ☐ Audit logging for security-relevant operations
- ☐ Tests for domain logic changes
- ☐ MDX integrity passes

Deploy-on-Merge

Merging to `main` triggers automatic Netlify deployment:

1. Merge PR, push to main
2. Netlify webhook fires
3. `pnpm build:netlify` runs
4. Site deployed to production (typically 2-4 minutes)

Server-Only Boundary

When adding new scoring, classification, or synthesis logic:

1. Place the file in `lib/server/` or `lib/decision/`
2. Add `import "server-only"` at the top
3. Export only public-safe types and functions
4. Never import server-only modules from Pages Router page components — use API routes instead

---

Chapter 42: Scripts Reference

Build & Development

Script	Command	Purpose
`dev`	MDX gate + PDF registry + CL build + Next.js dev	Start dev server
`dev:full`	Env check + concurrent CL watch + Next.js dev	Full dev server
`dev:windows`	Windows-compatible dev server	Windows dev
`build`	Generate EPUBs + Next.js build	Production build
`build:netlify`	MDX gate + CL clean + integrity + build:fast	Netlify-specific build
`build:fast`	PDF registry + Next.js build (7GB heap) + clean	Fast build
`build:analyze`	Build with webpack-bundle-analyzer	Analyze bundles
`clean`	Remove .next, .contentlayer, cache	Clean build
`clean:hard`	Deep clean (scripts/clean-project.js)	Deep clean

Content

Script	Command	Purpose
`mdx:gate`	`node scripts/mdx-illegal-jsx-gate.mjs`	Block illegal JSX in MDX
`mdx:integrity`	`node scripts/mdx-integrity-check.mjs`	Validate MDX component tags
`mdx:sanitize`	Sanitize all MDX files	Sanitize MDX
`content:validate`	`node scripts/validate-frontmatter.mjs`	Validate content frontmatter
`contentlayer:build`	`contentlayer2 build`	Build content layer
`contentlayer:clean`	Remove .contentlayer directory	Clean CL

PDF

Script	Purpose
`pdf:build:full`	Registry full + generate all + sync
`pdf:generate-all`	Unified generator (Puppeteer + LibreOffice)
`pdf:registry:build`	Generate `pdf-registry.generated.ts`
`pdf:audit`	Full audit (registry + links + dupes + governance)
`pdf:repair`	Auto-repair empty/corrupt PDFs
`pdf:validate`	Validate PDF file integrity
`pdf:stats`	Generate PDF statistics
`pdf:governance`	Verify PDF governance rules
`pdf:ebook`	Render EPUB format
`pdf:vault-pack`	Build vault download pack

Database

Script	Command	Purpose
`db:generate`	`prisma generate`	Generate Prisma client
`db:push`	`prisma db push`	Push schema without migration
`db:migrate`	`prisma migrate dev`	Create new migration
`db:deploy`	`prisma migrate deploy`	Apply migrations (production)
`db:seed`	`tsx prisma/seed.ts`	Seed database
`db:studio`	`prisma studio`	Open Prisma Studio GUI
`db:reset`	`prisma migrate reset --force && pnpm db:seed`	Reset and reseed
`db:status`	Check migration status	Status check
`db:backup`	Backup database	Backup
`db:restore`	Restore from backup	Restore

Vault

Script	Purpose
`vault:sync`	Master vault synchronization (8GB heap)
`vault:fix`	Repair vault issues
`vault:audit`	Audit vault integrity
`vault:manifest`	Generate vault manifest
`vault:ready`	CL safe + audit + glossary + check + manifest
`vault:deploy`	Audit + sync + build

Quality & Testing

Script	Command	Purpose
`quality:full`	`node scripts/quality/full-validation.mjs`	Full validation gate (8 checks)
`test:unit`	`vitest run`	Run unit + integration tests
`test:e2e`	Playwright	Run end-to-end tests
`test:coverage`	V8 coverage	Run with coverage
`lint`	`tsc --noEmit`	TypeScript check
`format`	Prettier write all files	Auto-format
`format:check`	Prettier check (CI mode)	CI format check
`typecheck`	`tsc --noEmit` (strict)	Full type check
`validate`	typecheck + lint + test	Full validation

Security & Checks

Script	Purpose
`security:audit`	pnpm audit (high severity)
`check:all`	health + system + security + content-boundary
`check:content-boundary`	Server/client import boundary

Fixes

Script	Purpose
`fix:all`	Run ALL fix scripts
`fix:windows`	Windows path normalization
`fix:mdx`	MDX encoding issues
`fix:encoding`	UTF-8/BOM issues
`fix:prisma`	Regenerate Prisma client
`fix:imports`	Fix broken import paths

Deploy & Environment

Script	Purpose
`deploy`	Validate links + commit + push
`env:check`	Check environment variables
`env:validate`	Full env validation
`health`	System health check
`verify:production`	Verify live production site

Assets

Script	Purpose
`assets:optimize`	Standard image optimization
`assets:enterprise`	Premium quality pipeline
`assets:enterprise:production`	Production AVIF generation
`optimize:premium`	Ultra-quality AVIF images

Audit & Monitoring

Script	Purpose
`audit:links`	Verify all internal/external links
`audit:email`	Email system integrity
`audit:pricing`	Pricing authority check
`audit:checkout`	Checkout flow verification
`audit:access`	Access control consistency
`stats`	PDF + assets + vault audit combined

Chapter 43: Troubleshooting Guide

Build Failures

MDX Integrity Failure:

```
Error: MDX integrity check failed for content/briefs/example.mdx
```

Fix:

```
pnpm fix:mdx          # Auto-repair encoding
pnpm mdx:sanitize     # Sanitize all MDX
pnpm mdx:integrity    # Re-verify
```

Contentlayer Cache Corruption:

```
Error: Cannot read properties of undefined (reading 'type')
```

Fix:

```
pnpm contentlayer:clean   # Remove .contentlayer/
pnpm contentlayer:build   # Rebuild from scratch
```

Memory Limit (OOM):

```
FATAL ERROR: Reached heap limit Allocation failed
```

Fix: Ensure `NODE_OPTIONS=--max-old-space-size=7168` is set.

Build Size Exceeded: Fix:

```
pnpm build:analyze   # Identify large chunks
```

Database Issues

Connection Refused:

```
Error: Can't reach database server
```

Fix: Verify `DATABASE_URL` in `.env`. Check network connectivity and connection string for Neon.

Migration Drift:

```
Error: The database schema is not in sync
```

Fix:

```
pnpm db:push    # Force schema sync (dev only)
pnpm db:reset   # Full reset + re-seed (destructive)
```

PDF Generation Failures

LibreOffice Not Found: Fix (Windows): Ensure LibreOffice is installed at `C:\Program Files\LibreOffice\program\soffice.com`.

Puppeteer Chrome Missing: Fix: `npx puppeteer browsers install chrome` or set `PUPPETEER_EXECUTABLE_PATH`.

PDF Size Validation (0 bytes): Fix: `pnpm pdf:repair`

Auth Issues

Secret Mismatch (`JWTVerificationFailed`): Fix: Ensure `NEXTAUTH_SECRET` is identical across all environments.

Callback URL Mismatch (`redirect_uri_mismatch`): Fix: Add the URL to OAuth provider's allowed redirect URIs. Ensure `NEXTAUTH_URL` matches the actual domain.

Content Issues

Frontmatter Validation: Fix: `pnpm content:validate` — shows exactly which fields are wrong.

Encoding Issues (Windows): Fix: `pnpm fix:encoding` — removes BOM from all files.

Windows Path Issues: Fix: `pnpm fix:windows` — normalizes all paths.

Server-Only Import in Pages Router

Problem: Build fails with "server-only" import error in a Pages Router page.

Cause: A page component imports a module that uses `import "server-only"`. The Pages Router does not support the `server-only` package in page components.

Fix: Move the server-only call to an API route. Only App Router Server Components can import `server-only` modules directly.

ioredis Client Bundling

Problem: Build fails or bundle size explodes with ioredis in the client bundle.

Fix: Redis imports must be dynamic (`await import(...)`) or isolated in server-only files. Use `lib/server/security/persistent-rate-limit.ts` as the entry point.

Deployment Issues

Bundle Size Too Large: Fix: Review `excluded_files` in `netlify.toml`. Run `pnpm build:analyze`.

Missing Environment Variables: Fix: Add variable in Netlify dashboard, redeploy.

Function Timeout: Fix: Optimize the function. Netlify functions have a 26-second timeout (Pro: 60s). Move heavy work to scheduled/background functions.

APPENDICES

Appendix A: Prisma Model Inventory

126 models in `prisma/schema.prisma`:

Model	Domain
Organisation	Enterprise
AlignmentCampaign	Enterprise
AlignmentSnapshot	Enterprise
OrganisationInvite	Enterprise
OrganisationMembership	Enterprise
GovernanceLog	Governance
User	Identity
Account	Identity
VerificationToken	Identity
Entitlement	Access
AccessKey	Access
AccessKeyUse	Access
AccessAuditLog	Audit
AccessInvite	Access
CampaignParticipant	Enterprise
AuditResponse	Enterprise
EnterpriseAssessment	Enterprise
TeamAssessmentSnapshot	Enterprise
TeamAssessmentCampaign	Enterprise
TeamAssessmentInvite	Enterprise
TeamAssessmentResponse	Enterprise
TeamAssessmentAggregate	Enterprise
OrganisationAssessmentSnapshot	Enterprise
LeadershipGapSnapshot	Enterprise
EnterpriseReport	Enterprise
DiagnosticRecord	Diagnostics
DiagnosticReportOrder	Diagnostics
DiagnosticArtifact	Diagnostics
DiagnosticRegenerationJob	Diagnostics
DiagnosticAuditEvent	Diagnostics
DiagnosticArtifactAccessGrant	Diagnostics
DiagnosticLineageEvent	Diagnostics

Model	Domain
ProofEvidence	Diagnostics
ClientEntitlement	Access
BillingCustomer	Commerce
OperationalIncident	Operations
ArtifactManifest	Operations
JobDeadLetter	Operations
ServiceLevelSnapshot	Operations
RunbookEntry	Operations
DecisionRecommendationSession	Decision
DecisionRecommendationImpression	Decision
DecisionRecommendationClick	Decision
DecisionRecommendationConversion	Decision
DecisionSessionFollowup	Decision
DecisionAssetEfficacy	Decision
DecisionAssetContextPerformance	Decision
GovernanceMetricDefinition	Governance
DecisionSignalRegistry	Decision
DecisionGovernanceAlert	Decision
PurposeAlignmentAssessment	Alignment
PurposeAlignmentReport	Alignment
PurposeAlignmentReminderPreference	Alignment
PurposeAlignmentReminderLog	Alignment
DealFlowSubmission	Commerce
DecisionAssetPerformance	Decision
InnerCircleMember	Access
AdminSession	Identity
ApiKey	Identity
ApiLog	Operations
InnerCircleKey	Access
MfaSetup	Identity
Session	Identity
SecurityLog	Security

Model	Domain
PageView	Analytics
RateLimitLog	Security
SystemAuditLog	Audit
UserPreference	Identity
UserIntegration	Identity
StrategyInquiry	Strategy
StrategyIntake	Strategy
ConstitutionalIntakeReport	Diagnostics
ExecutiveReportingRun	Enterprise
ExecutiveReportingArtifact	Enterprise
DiagnosticJourney	Diagnostics
DiagnosticEvidenceNode	Diagnostics
DiagnosticDecisionObject	Diagnostics
DecisionDependency	Decision
DecisionStakeholder	Decision
StakeholderPosition	Decision
AuditEvent	Audit
EnforcementPlaybook	Governance
PlaybookApplication	Governance
FoundationTelemetryEvent	Operations
RetainerContract	Commerce
RetainedDecision	Commerce
EnforcementCycle	Governance
DiagnosticStageRecord	Diagnostics
LongitudinalComparisonRecord	Diagnostics
MultiStakeholderResult	Diagnostics
OutcomeVerificationRecord	Diagnostics
DiagnosticThreadSnapshot	Diagnostics
MonitoringSnapshot	Operations
BenchmarkFact	Operations
BenchmarkCohortSnapshot	Operations
StrategyRoomSession	Strategy

Model	Domain
StrategyRoomRecommendationImpression	Strategy
StrategyRoomFollowup	Strategy
StrategyRoomConversion	Strategy
AdminDecisionContextualEfficacy	Decision
ContentMetadata	Content
Framework	Content
StrategicLink	Content
ContentRelation	Content
PrivateAnnotation	Content
CanonEntry	Content
StrategicFramework	Content
StrategicIntervention	Strategy
CorrectionNode	Governance
DecisionAssetGovernanceRule	Decision
DownloadAuditEvent	Commerce
PrintAsset	Content
PremiumDownloadToken	Commerce
PremiumDownloadAttempt	Commerce
FrameworkAccessLog	Content
DecisionJourneyEvent	Decision
StrategyRoomExecutionSession	Strategy
StrategyDecisionLog	Strategy
FailedEntitlementGrant	Access
ConsequenceTimeline	Decision
EscalationEvent	Governance
CalibrationState	Operations
CalibrationEvent	Operations
PatternBreakerContract	Governance
RateLimitBucket	Security
DecisionMemory	Decision

Appendix B: API Route Inventory

Routes under `pages/api/`

Route	Method	Auth Required
`/api/diagnostics/score`	POST	No
`/api/diagnostics/submit`	POST	No (rate-limited)
`/api/diagnostics/[ref]`	GET	Optional
`/api/diagnostics/list`	GET	Yes
`/api/diagnostics/report`	GET	Yes
`/api/diagnostics/create-report-checkout`	POST	Yes
`/api/diagnostics/constitutional-intake/*`	POST/GET	Varies
`/api/diagnostics/enterprise`	POST	Yes
`/api/diagnostics/executive-reporting`	POST	Yes
`/api/diagnostics/team-alignment`	POST	Yes
`/api/diagnostics/spine/*`	GET	Yes
`/api/diagnostics/outcomes/verify`	POST	Yes
`/api/diagnostics/reentry`	POST	Yes
`/api/diagnostics/longitudinal`	GET	Yes
`/api/auth/[...nextauth]`	GET/POST	No
`/api/auth/identity`	GET	No
`/api/auth/me`	GET	Yes
`/api/auth/session`	GET	No
`/api/auth/mint`	POST	Yes
`/api/auth/sovereign-login`	POST	Dev only
`/api/stripe/*`	POST	Webhook secret
`/api/health`	GET	No
`/api/contact`	POST	No (rate-limited)
`/api/subscribe`	POST	No (rate-limited)
`/api/downloads/*`	GET	Varies
`/api/inner-circle/*`	GET/POST	Yes (inner_circle+)
`/api/admin/*`	GET/POST/PUT/DELETE	Yes (admin+)
`/api/strategy-room/*`	GET/POST	Yes
`/api/ogr/*`	GET/POST	Yes

Route	Method	Auth Required
`/api/billing/*`	GET/POST	Yes

Public Routes (No Auth)

Method	Path	Description
GET	`/api/root`	API root/health
GET	`/api/v2/health`	Detailed health check
GET	`/api/search`	Public content search
POST	`/api/telemetry/global`	Global telemetry events
POST	`/api/telemetry/resonance`	Content resonance tracking
POST	`/api/pulse/submit`	Pulse survey submission
GET	`/api/stats`	Public statistics

Authentication Routes

Method	Path	Description
POST	`/api/auth/sovereign`	Sovereign auth flow
POST	`/api/sovereign/auth`	Sovereign login
POST	`/api/sovereign/logout`	Sovereign logout
GET	`/api/sovereign/history`	Auth history
POST	`/api/admin/dev-login`	Dev-only admin login

Inner Circle Routes

Method	Path	Description
POST	`/api/inner-circle/verify`	Verify IC membership
POST	`/api/inner-circle/issue`	Issue IC credential
GET	`/api/inner-circle/admin/export`	Export IC data (admin)

Enterprise & Alignment Routes

Method	Path	Description
GET/POST	`/api/alignment/enterprise`	Enterprise alignment
POST	`/api/alignment/enterprise/organisations`	Create organisation
GET/POST	`/api/alignment/enterprise/campaigns`	Campaign CRUD
GET/PUT	`/api/alignment/enterprise/campaigns/[id]`	Single campaign
POST	`/api/alignment/enterprise/campaigns/[id]/close`	Close campaign
POST	`/api/alignment/enterprise/campaigns/[id]/invite`	Invite participant
GET	`/api/alignment/enterprise/campaigns/[id]/report`	Campaign report
POST	`/api/alignment/enterprise/campaigns/[id]/aggregate`	Aggregate results
POST	`/api/alignment/enterprise/respond/[token]`	Submit response
GET	`/api/alignment/enterprise/assessments`	List assessments

Team Assessment Routes

Method	Path	Description
POST	`/api/team-assessment/campaign/create`	Create team campaign
POST	`/api/team-assessment/campaign/[id]/invites`	Send invites
POST	`/api/team-assessment/campaign/[id]/close`	Close assessment
GET	`/api/team-assessment/campaign/[id]/aggregate`	Team aggregate
GET	`/api/team-assessment/campaign/[id]/status`	Campaign status
POST	`/api/team-assessment/respond/[token]`	Submit response

Diagnostics Routes (Extended)

Method	Path	Description
POST	`/api/diagnostics/campaigns/[id]/aggregate`	Diagnostic aggregate
POST	`/api/diagnostics/multi-stakeholder`	Multi-user collision analysis

Decision Intelligence Routes

Method	Path	Description
POST	`/api/decision/guidance`	Decision guidance AI
GET	`/api/decision/metadata-audit`	Metadata audit
POST	`/api/interpret`	Content interpretation

Strategy Room Routes

Method	Path	Description
POST	`/api/strategy-room/session/init`	Initialize session
POST	`/api/strategy-room/session/impression`	Log impression
POST	`/api/strategy-room/session/click`	Log click
POST	`/api/strategy-room/session/conversion`	Log conversion
POST	`/api/strategy-room/session/followup`	Follow-up action
GET	`/api/strategy-room/results`	Session results
GET	`/api/strategy-room/execution/[id]`	Execution session
POST	`/api/strategy-room/execution/[id]/decisions`	Evaluate decision

Downloads & Vault Routes

Method	Path	Description
GET	`/api/download/[token]`	Token-gated download
GET	`/api/downloads/[slug]`	Slug-based download
GET	`/api/vault/status`	Vault status
GET	`/api/vault/[...slug]`	Vault content access

Reports & Export Routes

Method	Path	Description
GET	`/api/campaigns/[id]/report`	Campaign report
GET	`/api/campaigns/[id]/report/json`	JSON export
GET	`/api/campaigns/[id]/report/pdf`	PDF export
GET	`/api/sovereign/report`	Sovereign report
GET	`/api/purpose-alignment/report`	PA report
GET	`/api/purpose-alignment/report/[assessmentId]`	Single PA report
GET	`/api/executive-reporting/export/pdf`	Executive PDF export
GET	`/api/executive-reporting/export/boardroom-pdf`	Boardroom PDF
POST	`/api/executive-reporting/export/intervention`	Intervention export

Admin Routes (Founder/Admin Only)

Method	Path	Description
GET/POST	`/api/admin/campaigns`	Campaign management
GET/PUT	`/api/admin/campaigns/[id]`	Single campaign admin
GET	`/api/admin/campaigns/[id]/report`	Admin report view
GET	`/api/admin/campaigns/[id]/report/pdf`	Admin PDF report
GET	`/api/admin/campaigns/[id]/report/export-json`	Admin JSON export
GET	`/api/admin/decision/efficacy`	Decision efficacy
GET	`/api/admin/decision/contextual-efficacy`	Contextual efficacy
GET	`/api/admin/decision/contextual-ranking`	Context ranking
GET	`/api/admin/decision/governance`	Governance dashboard
GET	`/api/admin/decision/performance`	Performance metrics
GET	`/api/admin/decision/signal-registry`	Signal registry
POST	`/api/admin/decision/rebuild-efficacy`	Rebuild efficacy
POST	`/api/admin/decision/rebuild-contextual-efficacy`	Rebuild context
POST	`/api/admin/decision/rebuild-governance-alerts`	Rebuild alerts
POST	`/api/admin/decision/rebuild-performance`	Rebuild performance
GET	`/api/admin/decision-intelligence`	DI dashboard
GET	`/api/admin/commercial`	Commercial metrics
GET	`/api/admin/positioning`	Market positioning

Other Routes

Method	Path	Description
POST	`/api/audit/log`	Submit audit log
POST	`/api/audit/submit`	Audit form submission
POST	`/api/constitutional/appeal`	Constitutional appeal
GET	`/api/constitutional/audit`	Constitutional audit
POST	`/api/cron/snapshot`	Cron snapshot trigger
GET	`/api/entitlements`	User entitlements
POST	`/api/checkout`	Payment checkout
POST	`/api/analytics/journey`	Journey analytics
POST	`/api/leads/fuse`	Lead fusion
GET	`/api/predictive/insights/[campaignId]`	Predictive insights
POST	`/api/premium/forensics/attribution`	Attribution forensics
GET/PUT	`/api/purpose-alignment/reminders/preferences`	Reminder prefs
GET	`/api/v2/users`	User management

> **Auth Legend:** None = public, Token = URL token (respondent links), Auth = authenticated user session, Admin = ADMIN or OWNER role required.

Appendix C: Environment Variable Reference

Variable	Category	Required	Description
`NODE_ENV`	Application	Yes	Runtime environment
`NEXT_PUBLIC_APP_ENV`	Application	Yes	Client-visible environment
`NEXT_PUBLIC_APP_NAME`	Application	No	Display name
`NEXT_PUBLIC_APP_URL`	Application	Yes	Client-side base URL
`NEXT_PUBLIC_SITE_URL`	Application	Yes	Canonical site URL
`SITE_URL`	Application	Yes	Server-side site URL
`ALLOWED_ORIGINS`	Application	Yes	CORS whitelist (comma-separated)
`DATABASE_URL`	Database	Yes	PostgreSQL connection string (Neon)
`DIRECT_URL`	Database	Yes	Direct DB connection (bypasses pooler)
`REDIS_URL`	Redis	No	Redis connection string
`REDIS_DISABLED`	Redis	No	Disable Redis entirely
`USE_REDIS`	Redis	No	Enable Redis features
`UPSTASH_REDIS_REST_URL`	Redis	No	Upstash REST endpoint
`UPSTASH_REDIS_REST_TOKEN`	Redis	No	Upstash auth token
`NEXTAUTH_URL`	Auth	Yes	NextAuth base URL
`NEXTAUTH_SECRET`	Auth	Yes	JWT signing secret (min 32 chars)
`JWT_SECRET`	Auth	Yes	Additional JWT secret
`JWT_ALGORITHM`	Auth	No	JWT algorithm (default HS256)
`JWT_EXPIRES_IN`	Auth	No	Token expiry (default 30d)
`ENCRYPTION_KEY`	Auth	Yes	General encryption key
`CSRF_SECRET`	Auth	Yes	CSRF token secret
`ACCESS_COOKIE_SECRET`	Auth	Yes	Cookie encryption secret
`SESSION_COOKIE_PREFIX`	Auth	No	Cookie name prefix (default aol)
`ADMIN_JWT_SECRET`	Admin	Yes	Admin-specific JWT
`ADMIN_API_KEY`	Admin	Yes	Admin API authentication
`ADMIN_SECRET_TOKEN`	Admin	Yes	Admin token validation
`ADMIN_SECRET`	Admin	Yes	Admin secret
`ADMIN_USER_EMAIL`	Admin	Yes	Bootstrap admin email
`ADMIN_USER_PASSWORD`	Admin	Yes	Bootstrap admin password
`ADMIN_ALLOWED_EMAILS`	Admin	Yes	Admin email whitelist

Variable	Category	Required	Description
`CRON_SECRET`	Cron	Yes	Cron endpoint auth
`INTERNAL_BYPASS_KEY`	Cron	Yes	Internal service bypass
`AUDIT_EDGE_SECRET`	Cron	Yes	Audit edge function auth
`AOL_BRAND_NAME`	Brand	No	Brand display name
`AOL_HASH_SALT`	Brand	Yes	General hashing salt
`AOL_SESSION_TTL_DAYS`	Brand	No	Session lifetime (default 30)
`AOL_TOKENSTORE_BACKEND`	Brand	No	Token storage backend (default postgres)
`SYSTEM_INTEGRITY_SALT`	Brand	Yes	Vault/PDF integrity salt
`ANONYMITY_SALT`	Brand	Yes	Identity anonymization
`DENYLIST_PEPPER`	Brand	Yes	Denylist hash pepper
`EMAIL_PROVIDER`	Email	No	Email service provider (default resend)
`EMAIL_FROM`	Email	No	From address
`RESEND_API_KEY`	Email	Partial	Resend API key
`CONTACT_RECEIVER_EMAIL`	Email	No	Contact form recipient
`INNER_CIRCLE_STORE`	Inner Circle	Yes	IC data store (default postgres)
`INNER_CIRCLE_DB_URL`	Inner Circle	Yes	IC database URL
`INNER_CIRCLE_JWT_SECRET`	Inner Circle	Yes	IC JWT secret
`INNER_CIRCLE_KEY_SECRET`	Inner Circle	Yes	IC key encryption
`ENTERPRISE_ALIGNMENT_INVITE_SECRET`	Enterprise	Yes	Enterprise invite HMAC
`DIAGNOSTIC_HMAC_SECRET`	Enterprise	Yes	Diagnostic integrity
`DIAGNOSTIC_WATERMARK_SECRET`	Enterprise	Yes	PDF watermark secret
`DIAGNOSTIC_STORAGE_PROVIDER`	Enterprise	No	Storage backend (default local)
`OGR_SESSION_SECRET`	Sovereign	Yes	OGR session signing
`OGR_SOVEREIGN_KEY`	Sovereign	Yes	Sovereign access key
`SOVEREIGN_ACCESS_KEY`	Sovereign	Yes	Sovereign auth
`STRIPE_SECRET_KEY`	Payments	No	Stripe API key
`STRIPE_PUBLISHABLE_KEY`	Payments	No	Stripe publishable key
`STRIPE_WEBHOOK_SECRET`	Payments	No	Stripe webhook validation
`OPENAI_API_KEY`	AI	No	OpenAI API key
`GOOGLE_OAUTH_CLIENT_ID`	OAuth	No	Google OAuth client
`GOOGLE_OAUTH_CLIENT_SECRET`	OAuth	No	Google OAuth secret

Variable	Category	Required	Description
`OAUTH_TOKEN_ENCRYPTION_KEY`	OAuth	No	Token encryption (min 32 chars)
`PDF_OUTPUT_DIR`	PDF	No	PDF output directory
`PDF_TEMP_DIR`	PDF	No	PDF temp directory
`PDF_FONTS_DIR`	PDF	No	Font directory
`ARTIFACT_ACCESS_SECRET`	PDF	Yes	Artifact access signing
`DOWNLOAD_TOKEN_SECRET`	PDF	Yes	Download token HMAC
`RATE_LIMIT_MAX_REQUESTS`	Security	No	Max requests per window
`RATE_LIMIT_WINDOW_MS`	Security	No	Window duration (ms)
`ENABLE_ANALYTICS`	Feature Flags	No	Analytics enabled
`ENABLE_PDF_GENERATION`	Feature Flags	No	PDF generation enabled
`ENABLE_EMAIL_NOTIFICATIONS`	Feature Flags	No	Email sending enabled
`SKIP_DB`	Feature Flags	No	Skip database operations
`LOG_LEVEL`	Feature Flags	No	Logging verbosity
`DEBUG_CONTENTLAYER`	Development	No	Verbose CL logging
`IS_WINDOWS`	Development	No	Windows compatibility mode

Appendix D: Database Schema Extended Reference

Model Breakdown by Domain

Core Identity & Access

Model	Purpose	Key Relations
`User`	Platform user	→ Account, Entitlement, AccessKeyUse
`Account`	OAuth account link	→ User
`VerificationToken`	Email verification	Standalone
`Entitlement`	Access grants	→ User
`AccessKey`	Redeemable access codes	→ AccessKeyUse
`AccessKeyUse`	Redemption log	→ User, AccessKey
`AccessAuditLog`	Access attempt log	Standalone
`AccessInvite`	Invitation tokens	Standalone

Organisation & Enterprise

Model	Purpose	Key Relations
`Organisation`	Enterprise client	→ Memberships, Campaigns, Invites
`AlignmentCampaign`	Assessment campaign	→ Organisation, Participants
`AlignmentSnapshot`	Campaign results	→ Campaign, Organisation
`OrganisationInvite`	Org join invitation	→ Organisation
`OrganisationMembership`	User-org binding	→ Organisation, Campaigns
`CampaignParticipant`	Campaign member	→ Campaign, Membership
`EnterpriseAssessment`	Individual assessment	→ Campaign
`EnterpriseReport`	Generated report	→ Campaign

Diagnostics

Model	Purpose	Key Relations
`DiagnosticRecord`	Diagnostic instance	→ Artifacts, Orders
`DiagnosticReportOrder`	Payment for report	→ DiagnosticRecord
`DiagnosticArtifact`	Generated PDF/artifact	→ DiagnosticRecord
`DiagnosticRegenerationJob`	Report rebuild queue	→ DiagnosticRecord
`DiagnosticAuditEvent`	Diagnostic audit trail	→ DiagnosticRecord
`DiagnosticJourney`	Multi-stage diagnostic	→ Evidence, Decisions
`DiagnosticEvidenceNode`	Evidence data point	→ Journey
`DiagnosticDecisionObject`	Decision record	→ Journey

Decision Intelligence

Model	Purpose
`DecisionRecommendationSession`	AI recommendation session
`DecisionRecommendationImpression`	Content shown to user
`DecisionRecommendationClick`	User click tracking
`DecisionRecommendationConversion`	Conversion event
`DecisionSessionFollowup`	Follow-up actions
`DecisionAssetEfficacy`	Asset performance metrics
`DecisionAssetContextPerformance`	Context-specific performance
`DecisionSignalRegistry`	Signal definitions
`DecisionGovernanceAlert`	Governance violations
`DecisionAssetGovernanceRule`	Asset-level rules

Governance & Audit

Model	Purpose	Key Fields
`GovernanceLog`	Admin action log	action, performedBy, target
`SecurityLog`	Security events	event, ip, userId
`SystemAuditLog`	System audit trail	actorType, action, severity
`AuditEvent`	Generic audit event	type, severity, metadata
`RateLimitLog`	Rate limit violations	ip, endpoint, count

Inner Circle & Membership

Model	Purpose
`InnerCircleMember`	IC membership record
`InnerCircleKey`	IC access key
`AdminSession`	Admin login session
`ApiKey`	API key management
`ApiLog`	API usage logging
`MfaSetup`	MFA configuration
`Session`	User session

Content & Assets

Model	Purpose
`ContentMetadata`	Content piece metadata
`Framework`	Strategic framework definition
`StrategicLink`	Framework relationships
`ContentRelation`	Content cross-references
`PrivateAnnotation`	Private content notes
`CanonEntry`	Canon/lexicon entry
`StrategicFramework`	Framework configuration
`PrintAsset`	Print-ready asset
`DownloadAuditEvent`	Download tracking
`PremiumDownloadToken`	Gated download token
`PremiumDownloadAttempt`	Download attempt log

Commercial

Model	Purpose
`DealFlowSubmission`	Inbound deal flow
`StrategyInquiry`	Strategy inquiry form
`StrategyIntake`	Intake assessment
`RetainerContract`	Client retainer
`RetainedDecision`	Decision under retainer
`BillingCustomer`	Payment customer
`ClientEntitlement`	Client-specific access

Enums (34 Total)

```
AlignmentBand, AlignmentDomain, CorrectionStatus, MemberRole,
AccessTier, MemberStatus, ContentType, LinkType, AnnotationPriority,
InquiryStatus, StrategyIntakeStatus, DownloadEventType,
DownloadDeliveryMode, DownloadContentType, AccessType, AuditSeverity,
SecurityEvent, HttpMethod, SessionStatus, KeyStatus, MfaMethod,
Permission, DiagnosticSeverity, DiagnosticLifecycleStatus,
DiagnosticReportStatus, DiagnosticArtifactKind,
DiagnosticStorageProvider, DiagnosticRegenerationStatus,
GovernanceSeverity, GovernanceStatus, UserRole, EntitlementType,
EntitlementStatus, AccessKeyStatus, InviteStatus, AuditActorType
```

Appendix E: Quality Gate Checklist

Before any merge to main, verify:

- [ ] `git status` — no unexplained untracked or modified files
- [ ] `npx prisma validate` — schema is valid
- [ ] `npx prisma generate` — client is generated
- [ ] `npx tsc --noEmit` — zero TypeScript errors, no exclusions
- [ ] `pnpm pdf:audit` — PDF audit passes
- [ ] `node scripts/mdx-integrity-check.mjs` — MDX integrity passes
- [ ] `node scripts/mdx-illegal-jsx-gate.mjs` — MDX gate passes
- [ ] `npx vitest run` — full suite passes (427 test files)
- [ ] `next build --webpack` — build succeeds
- [ ] No `server-only` imports in client-bundled code
- [ ] No scoring logic, thresholds, or classification rules in client bundle
- [ ] No secrets in committed files

Or run all at once:

```
node scripts/quality/full-validation.mjs
```

Appendix F: Architecture Decision Records

ADR-001: Pages Router Primary, App Router for Admin

Field	Detail
Status	Accepted
Context	Platform uses both routing systems. Pages Router handles the primary user-facing experience (diagnostics, content, auth, commerce). App Router handles admin, enterprise, and PDF rendering.
Decision	Pages Router (`pages/`) is the primary router for user-facing routes. App Router (`app/`) is used for admin dashboards, enterprise campaign management, PDF rendering, and purpose alignment.
Rationale	Pages Router provides stable, well-tested patterns for the core commercial surface. App Router provides Server Components and nested layouts for admin/enterprise where they add value.
Consequences	Two routers coexist. Engineers must understand which router handles which concerns. The middleware behaviour differs between routers.

ADR-002: Prisma over raw SQL / Drizzle

Field	Detail
Status	Accepted
Context	Need type-safe ORM for complex schema (126 models). Schema complexity and team velocity prioritised.
Decision	Prisma 6.6 with Neon serverless adapter for production. PostgreSQL only — no SQLite in production.
Rationale	Generated types eliminate runtime type errors. Migration system provides auditable schema history. Prisma Studio enables rapid debugging. Serverless adapter eliminates connection pooling complexity.
Consequences	Bundle size impact (mitigated by serverless function isolation). Query limitations for complex aggregations (addressed with `$queryRaw` escape hatch).

ADR-003: Deterministic scoring over pure AI classification

Field	Detail
Status	Accepted
Context	Diagnostic results must be defensible and auditable. Pure AI classification introduces non-determinism.
Decision	AI generates synthesis and natural-language explanation. The arbiter system validates and classifies deterministically. AI cannot override classification.
Rationale	Regulatory defensibility (results reproducible given same evidence). Trust metric depends on consistency. Enterprise clients require auditability. Deterministic classification matches expert human rating at 94% agreement vs. 78% for pure AI.
Consequences	More engineering effort to maintain dual system (AI + arbiter). Some expressiveness lost (AI insights bounded by arbiter rules).

ADR-004: React State Only — No Global State Libraries

Field	Detail
Status	Accepted
Context	The system's complexity lives in the server-side domain layer, not the client. Client components collect input and render public DTOs.
Decision	React built-in state (`useState` , `useEffect` , `useReducer`) is the sole client state mechanism. No Zustand, Redux, Jotai, or React Query.
Rationale	Server-side scoring means the client is submit-only. No complex client state graphs needed. Eliminates dependency weight and conceptual overhead. sessionStorage + React state handles the limited client needs.
Consequences	Engineers accustomed to global state libraries must adapt to simpler patterns. Complex multi-step client flows (if needed) use `useReducer` .

ADR-005: Netlify over Vercel for deployment

Field	Detail
Status	Accepted
Context	Need serverless functions, scheduled tasks, and PDF generation with large binary dependencies.
Decision	Netlify with `@netlify/plugin-nextjs` for primary deployment.
Rationale	Function directory model provides clear separation. Built-in scheduling eliminates external scheduler. 250MB function bundle limit accommodates Puppeteer. Email plugin integration.
Consequences	Some Next.js features need adaptation. Must use `@netlify/plugin-nextjs` for compatibility — version pinning critical.

Appendix G: Revision History

Version	Date	Author	Changes
1.0	April 2026	Engineering Lead	Initial release (Parts A, B, C)
2.0	April 2026	Engineering Lead	Merged manual. Architecture updates: server-side scoring, PostgreSQL-only, React state only, Pages Router primary, Product Elevation Layer, Intelligence Spine contract, 6-tier severity target, 13-archetype target
3.0	May 2026	Engineering Lead	ZTHVF security convergence. Replaced middleware.ts with proxy.ts V5.1 architecture. Added Chapter 32A (ZTHVF Security Validation Framework). Updated rate limiting to match proxy configuration. IP exposure purge across 74 source files. PDF quarantine (84 PDFs moved to private_storage). Dependency audit closeout (hono >=4.12.16). Client bundle audit scoped to .next/static only. Red-team hostile suite expanded to 31 test cases
3.1	8 May 2026	Engineering Lead	Whole-ladder evidence spine closure. Added evidence persistence for Purpose Alignment (competingObligation, consequence, weakestDomain, strongestDomain, primaryPattern, contradictions, compositeScore, profile). Added financial exposure persistence (userCostOfDelayText, estimatedFinancialExposure, exposureBand, exposureBasis, costOfInaction projections). Created governed memory system with source-labelled, dated, safety-checked rendering via GovernanceEvidenceCarryForward across all 8 downstream surfaces (Executive Reporting, Strategy Room Entry, Strategy Room Session, Return Brief, Decision Centre, Oversight Brief, Control Room). Created whole-ladder evidence spine closure register (80/80 fields closed). Updated engineering manual with evidence spine architecture documentation.

Appendix H: Amendment Procedure

1. **Proposal** — Submit change request to Engineering Lead with rationale
2. **Review** — Founder reviews all changes to security or architecture sections
3. **Testing** — Changes validated against running system
4. **Approval** — Founder sign-off required for any security/access modifications
5. **Publication** — Version incremented, date updated, changelog entry added
6. **Communication** — All engineering staff notified of changes

> **KEY PRINCIPLE**
>
> This manual is a living document but NOT a wiki. Changes require formal review and approval. The manual reflects the system as it IS, not as someone wishes it were. If the system changes, the manual is updated. If the manual says something the system does not do, the system is wrong.

CLOSING DECLARATION

This document constitutes the complete, authoritative, and binding engineering reference for the Abraham of London platform.

Every architectural decision, security control, deployment configuration, and operational procedure documented herein is the product of deliberate design. They are not suggestions. They are not starting points for discussion. They are the standard.

Any engineer working on this platform — internal, contracted, or automated — is bound by the protocols, conventions, and prohibitions contained in this manual. Deviation without explicit written authorization from the Founder constitutes a breach of engineering discipline.

> **WARNING**
>
> No AI assistant, code generator, or automated tool may override, ignore, or "improve upon" the standards in this manual. If an AI suggests a pattern that contradicts this document, this document wins. Always. The manual is the final authority on how this system is built, deployed, secured, and operated.

This is not a 5-year document. This is a 100-year document. The tools will change. The frameworks will evolve. But the principles of disciplined engineering — validate inputs, enforce access, log everything, fail safely, deploy with confidence — are permanent.

Build accordingly.

END OF ENGINEERING MANUAL

Abraham of London — Decision Authority Infrastructure Version 3.1 — 8 May 2026